Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования
АМУРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
(ФГБОУ ВО «АмГУ»)

Институт компьютерных и инженерных наук
Кафедра математического анализа и моделирования
Направление подготовки 01.04.02 – Прикладная математика и информатика

КУРСОВОЙ ПРОЕКТ

на тему: «Анализ влияния крупномасштабных боевых действий на экономическую динамику в EVE Online с применением инструментов Python»

по дисциплине «Прикладные пакеты анализа данных»

Исполнитель
студент группы 5101 ом _____ Е.И. Зубко
(подпись, дата)

Руководитель
доцент, канд. физ.-мат. наук _____ Е.М. Веселова
(подпись, дата)

Нормоконтроль
доцент, канд. физ.-мат. наук _____ Е.М. Веселова
(подпись, дата)

Благовещенск 2025

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
АМУРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
(ФГБОУ ВО «АмГУ»)

Институт компьютерных и инженерных наук

Кафедра математического анализа и моделирования

З А Д А Н И Е

К курсовому проекту студента Зубко Евгения Ильича

1. Тема курсового проекта: «Анализ влияния крупномасштабных боевых действий на экономическую динамику в EVE Online с применением инструментов Python»

2. Срок сдачи студентом законченной работы: 24.12.2025 г.

3. Исходные данные к курсовому проекту: формулировка темы работы, литературные источники, ежемесячные отчёты, справочные данные и характеристики, инструменты для анализа данных

4. Содержание курсового проекта (перечень подлежащих разработке вопросов): анализ и формализация сведений о предметной области – описание экономического объекта и постановка задачи статистического моделирования; сбор, предобработка и консолидация ежемесячных отчётов;разработка воспроизводимых скриптов на языке python; проверка статистических гипотез

5. Перечень материалов приложения: исходный код программы анализа

6. Дата выдачи задания: 10.09.2025 г.

Руководитель курсового проекта:

Веселова Елена Михайловна, доцент, канд. физ.-мат. наук, доцент.

Задание принял к исполнению (10.09.2025): _____

РЕФЕРАТ

Курсовой проект содержит 59 страниц, 5 таблиц, 10 рисунков и 3 приложения

СТАТИСТИЧЕСКИЙ АНАЛИЗ, ЭКОНОМИЧЕСКАЯ ДИНАМИКА, КОРРЕЛЯЦИОННЫЙ АНАЛИЗ, РЕГРЕССИОННОЕ МОДЕЛИРОВАНИЕ, ВРЕМЕННЫЕ РЯДЫ, EVE ONLINE, ВИРТУАЛЬНАЯ ЭКОНОМИКА.

Данный курсовой проект посвящен разработке и применению методологии статистического анализа для оценки влияния крупномасштабных боевых действий на динамику ключевых экономических показателей сложной системы на примере массовой многопользовательской онлайн-игры EVE Online. Актуальность исследования обусловлена уникальными возможностями виртуальных экономик как источников полных, согласованных и детализированных данных, что позволяет отрабатывать методы анализа, применимые и к реальным экономическим процессам.

В работе решались следующие задачи: формирование согласованного аналитического датасета из разрозненных ежемесячных экономических отчетов (MER) за период с декабря 2019 по август 2025 года; разработка объективного бинарного индикатора военных периодов на основе статистического анализа распределения боевых потерь; проведение разведочного анализа данных; а также статистическая проверка выдвинутых гипотез.

Методологическую основу исследования составили методы описательной статистики, визуализации временных рядов, корреляционный анализ, методы сравнения выборок и регрессионное моделирование. Все расчеты были автоматизированы с использованием языка программирования Python и библиотек pandas, scipy.stats и statsmodels.

СОДЕРЖАНИЕ

Введение 5
1 Теоретические основы анализа экономической динамики в виртуальных мирах 7
1.1 Виртуальные экономики как источник данных для статистического моделирования 7
1.2 Методы статистического анализа, применяемые в исследовании 8
1.3 Связь методов с гипотезами и данными исследования 9
2 Источники данных и методология их подготовки 11
2.1 Характеристика исходных данных и их источников 11
2.2 Проблемы согласованности и разнородности данных 12
2.3 Предварительная обработка данных 12
2.4 Формирование итогового аналитического датасета 13
2.5 Методика выделения военных периодов 14
2.6 Ограничения и допущения 15
3 Эмпирический анализ влияния боевых действий на экономические показатели EVE online 16
3.1 Предварительная визуализация и описательная статистика динамики показателей 16
3.2 Статистическая проверка выдвинутых гипотез 23
3.3 Сводка результатов и интерпретация выявленных закономерностей 27
Заключение 29
Библиографический список 31
Приложение А Листинг программы на языке Python для подготовки данных для анализа 32
Приложение Б Листинг программы на языке Python для проведения разведочного анализа данных 40
Приложение В Листинг программы на языке Python для проверки установленных гипотез 48

ВВЕДЕНИЕ

Актуальность данной работы обусловлена ростом интереса к анализу сложных систем с полными данными. Экономики массовых онлайн-игр (MMO), такие как EVE Online, генерируют детализированные временные ряды, отражающие производство, торговлю и потребление. Это создаёт уникальную основу для применения современных методов статистики и анализа данных с целью изучения реакции системы на масштабные экзогенные события, что способствует отработке методологии, применимой и к менее прозрачным реальным процессам [1].

Цель работы – разработать и продемонстрировать методологию статистического анализа для оценки влияния дискретных экстремальных событий (крупномасштабных конфликтов) на динамику ключевых показателей сложной системы на примере данных EVE Online.

Задачи работы:

- сформировать согласованный датасет из разрозненных ежемесячных экономических отчётов EVE Online;
- разработать и обосновать бинарный индикатор («война»/«мир») на основе анализа распределения данных об уничтоженных активах;
- провести разведочный анализ данных (EDA) с визуализацией для выявления трендов и аномалий;
- проверить выдвинутые гипотезы с помощью корреляционного анализа, сравнения выборок (t-тест, U-тест) и регрессионного моделирования;
- разработать воспроизводимый скрипт на Python, автоматизирующий полный цикл анализа.

Гипотезы исследования:

- средний объём производства в периоды войны статистически значимо выше, чем в мирное время;

- объёмы торговли и скорость обращения денег имеют положительную корреляцию с уровнем боевых потерь.
- объём добычи ресурсов положительно коррелирует с показателем активности игроков.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АНАЛИЗА ЭКОНОМИЧЕСКОЙ ДИНАМИКИ В ВИРТУАЛЬНЫХ МИРАХ

1.1 Виртуальные экономики как источник данных для статистического моделирования

Современные сложные системы, будь то социальные сети, финансовые рынки или многопользовательские онлайн-игры, генерируют огромные массивы цифровых следов. Эти данные, представляющие собой детализированные временные ряды, фиксирующие действия агентов, являются основой для прикладного статистического анализа. В этом контексте виртуальные миры с экономикой полного цикла, такие как EVE Online, представляют собой уникальные «лабораторные стенды». Их ключевое отличие от реальных экономических систем – полная наблюдаемость и отслеживаемость всех значимых транзакций: от добычи единицы руды до уничтожения космического корабля в бою и последующей продажи его обломков [1].

Данные, извлекаемые из таких систем, обладают несколькими характеристиками, делающими их особенно ценными для исследования:

полнота: отчёты (например, Monthly Economic Reports от CCP) содержат агрегированные данные по всей совокупности игроков, а не выборочные обследования;

согласованность: показатели измеряются в единых стандартизированных единицах (ISK, кубические метры), что устраняет проблему сопоставимости;

наличие экзогенных шоков: крупные игровые события (войны) выступают в роли естественных экспериментов, позволяя изучать реакцию системы на резкое изменение условий.

Таким образом, работа с данными виртуальной экономики позволяет отработать полный цикл статистического исследования – от первичного

сбора и очистки данных до проверки сложных гипотез – на примере, свободном от многих «шумов» и ограничений реальных данных.

1.2 Методы статистического анализа, применяемые в исследовании

Для проверки сформулированных гипотез и анализа взаимосвязей между переменными, в работе применяется последовательность классических статистических методов, образующих аналитический конвейер.

1.2.1 Разведочный анализ данных и визуализация

Это первый этап, направленный на понимание структуры данных, выявление паттернов, аномалий и трендов. Ключевые инструменты:

анализ основных описательных статистик: расчёт среднего, медианы, стандартного отклонения, минимума и максимума для каждого временного ряда;

визуализация временных рядов: построение линейных графиков для анализа динамики и наложения периодов войн в виде затенённых областей;

сравнение распределений: использование боксплотов для наглядного сравнения распределения ключевых показателей в периоды войны и мира.

1.2.2 Методы проверки статистических гипотез

Для количественной проверки утверждений о различиях между группами используются критерии проверки гипотез:

t-критерий Стьюдента для независимых выборок: применяется для сравнения средних значений двух групп (война/мир) при условии нормальности распределения данных и равенства дисперсий.

U-критерий Манна-Уитни: непараметрический аналог t-критерия, используемый, когда предположение о нормальности нарушается. Он проверяет гипотезу о том, что распределения двух групп являются стохастически одинаковыми.

1.2.3 Корреляционный анализ

Метод для оценки силы и направления линейной или монотонной связи между двумя количественными переменными:

коэффициент корреляции Пирсона (r): измеряет силу линейной связи. Применяется к данным, приближающимся к нормальному распределению;

коэффициент ранговой корреляции Спирмена (ρ): оценивает монотонную связь (возрастающую или убывающую), не требующую линейности и нормальности данных. Критически важен для работы с экономическими показателями, часто имеющими нелинейные распределения.

1.2.4 Регрессионный анализ

Метод для моделирования зависимости между одной зависимой переменной и одной или несколькими независимыми переменными.

Множественная линейная регрессия: позволяет оценить изолированное влияние фактора войны на целевой показатель, контролируя при этом другие потенциально влиятельные факторы (например, общий тренд или сезонность). Анализ статистической значимости коэффицинтов регрессии (p-value) и силы модели (R^2) даёт количественную оценку взаимосвязей.

1.3 Связь методов с гипотезами и данными исследования

Выбор представленного набора методов напрямую обусловлен задачами и природой данных:

гипотеза о стимулировании производственной активности: утверждается, что средний уровень производственных показателей в периоды интенсивных конфликтов статистически значимо превышает средний уровень в периоды стабильности. Проверяется методами сравнения средних;

гипотеза об активизации торговых и денежных потоков: утверждается о наличии статистически значимой положительной связи между интенсивностью конфликтов и показателями торгового оборота, а также скорости обращения игровой валюты. Проверяется методами корреляционного и регрессионного анализа;

гипотеза о связи добывающей активности с численностью участников: утверждается о наличии статистически значимой положительной связи между объёмами добычи ресурсов и показателем общей активности

пользователей системы. Проверяется методами корреляционного и регрессионного анализа с учётом временного тренда.

Использование именно этого методологического стека на основе библиотек Python (pandas, scipy.stats, statsmodels) обеспечивает необходимую строгость, воспроизводимость и наглядность анализа, превращая сырые игровые данные в статистически обоснованные выводы о динамике сложной системы [2-4].

2 ИСТОЧНИКИ ДАННЫХ И МЕТОДОЛОГИЯ ИХ ПОДГОТОВКИ

2.1 Характеристика исходных данных и их источников

Основным источником данных для исследования послужили официальные ежемесячные экономические отчёты (Monthly Economic Reports, MER), публикуемые разработчиками игры CCP Games за период с декабря 2019 года по август 2025 года. Эти отчёты представляют собой структурированные наборы данных в формате CSV, организованные в виде отдельных директорий для каждого месяца. Каждая директория содержит от 30 до 50 файлов с различными категориями экономической статистики.

Таблица 1 – Основные файлы данных и их содержание

Категория данных	Имена файлов	Ключевые переменные	Назначение в исследовании
Макроэкономические агрегаты	ProducedDestroyedMined.csv,produced_destroyed_mined.csv	produced,destroyed,mining.value(старыйформат) production_isk,destruction_isk,mining_isk(новыйформат)	Проверка гипотезы о производстве
Торговая статистика	RegionalStats.csv, regional_stats.csv	trade_value, exports, imports	Проверка гипотезы о торговле
Боевые потери	kill_dump.csv, Killdump.csv	isk_destroyed, iskDestroyed	Классификация военных периодов
Денежный оборот	MoneySupply.csv, money_supply.csv	isk_velocity,total_isk	Анализ денежной массы
Добыча ресурсов	MiningHistoryBySecurityBand.csv,mining_history_by_security_band.csv	asteroid_volume_mined,gas_volume_mined,ice_volume_mined,moon_volume_mined	Проверка гипотезы о добыче

2.2 Проблемы согласованности и разнородности данных

В процессе работы с исходными данными были выявлены следующие проблемы согласованности:

2.2.1 Неоднородность форматов файлов

- Различия в именовании файлов: В разные периоды использовались различные названия. Например, файл с данными о производстве мог называться как `ProducedDestroyedMined.csv`, так и `produced_destroyed_mined.csv`;
- Различия в структуре данных: В файлах, относящихся к периоду до 2022 года, данные часто представлялись в дневном разрезе, тогда как в более поздних отчётах использовалась месячная агрегация.

2.2.2 Изменения в структуре переменных

- Разные названия столбцов: например, показатель уничтоженного имущества в разные периоды назывался `isk_destroyed`, `iskDestroyed`, `destroyed_value`.
- Изменение единиц измерения: Некоторые показатели в ранних отчётах имели другую размерность или методологию расчёта.

2.2.3 Неполнота данных

- Для некоторых месяцев отсутствовали отдельные категории данных (например, данные об объёме добычи за октябрь 2021 года).
- Временные ряды имели пропуски для отдельных месяцев (например, отсутствовал январь 2023 года).

2.3 Предварительная обработка данных

Для решения выявленных проблем и подготовки данных к анализу была разработана многоэтапная процедура предобработки.

Был разработан специализированный скрипт на Python, который выполнял следующие операции:

- рекурсивный обход директорий: Скрипт автоматически идентифицировал все папки с отчётами, начиная с декабря 2019 года;

- интеллектуальный поиск файлов: Реализован алгоритм, который находил файлы независимо от их именования (учёт различных регистров, форматов CamelCase/snake_case);
- адаптивное чтение данных: для каждого файла определялся разделитель (запятая или точка с запятой), кодировка и структура данных.

Для согласования данных с разной периодичностью применялись следующие методы:

- агрегация дневных данных: для файлов с дневной детализацией выполнялось суммирование значений по месяцам;
- единая временная сетка: Все данные были приведены к единой месячной периодичности;
- обработка пропусков: для отсутствующих месяцев использовалась линейная интерполяция на основе соседних значений;
- унификация названий столбцов: все переменные были приведены к единому формату именования;
- нормализация единиц измерения: Все стоимостные показатели выражены в ISK, объёмные показатели - в кубических метрах;
- проверка согласованности: Выполнена верификация взаимосвязей между показателями (например, проверено, что сумма экспорта и импорта соответствует общему объёму торговли).

2.4 Формирование итогового аналитического датасета

В результате процедуры предобработки был сформирован единый датасет, содержащий 67 наблюдений (месяцев) за период с декабря 2019 года по август 2025 года.

Таблица 2 – Структура итогового датасета

Переменная	Тип данных	Описание	Источник
history_date	datetime	Дата	Извлечено из имени
production_isk	float64	Стоимость произведенных товаров	ProducedDestroyedMined.csv

Продолжение таблицы 2

destruction_isk	float64	Стоимость уничтоженного имущества	ProducedDestroyedMined.csv
mining_isk	float64	Стоимость добытых ресурсов	ProducedDestroyedMined.csv
trade_value	float64	Общий объём торговли	RegionalStats.csv
total_exports	float64	Суммарный экспорт	RegionalStats.csv
total_imports	float64	Суммарный импорт	RegionalStats.csv
total_isk_destroyed	float64	Сумма боевых потерь	kill_dump.csv
isk_velocity	float64	Скорость обращения денежной массы	MoneySupply.csv
total_isk	float64	Общая денежная масса в экономике	MoneySupply.csv
total_volume_mined	float64	Физический объём добычи	MiningHistoryBySecurityBand.csv
is_war_period	int64	Бинарный индикатор военного периода	Рассчитано на основе total_isk_destroyed

2.5 Методика выделения военных периодов

Для перехода от качественного понятия «крупный военный конфликт» к количественно измеряемому критерию был использован агрегированный показатель месячного объёма боевых потерь. Алгоритм выделения соответствующих периодов был основан на статистическом анализе распределения этого показателя. В качестве объективного порога, отделяющего месяцы с экстремально высокой боевой активностью от фоновых колебаний, был выбран 75-й процентиль (верхняя квартиль) распределения. Это означает, что пороговым считается значение, превышаемое только в 25% наблюдений за весь исследуемый период.

Месяцы, в которых объём потерь превышал данное пороговое значение, классифицировались как военные, остальные - как мирные. Такой подход, основанный на статистической границе, а не на субъективной оценке, позволил выделить периоды действительно значимой конфликтной активности и минимизировать влияние случайных всплесков. В результате применения этого метода было идентифицировано 17 военных месяцев, что составляет 25,4% от всех 67 месяцев наблюдения с декабря 2019 года по август 2025 года.

2.6 Ограничения и допущения

В процессе подготовки данных были приняты следующие допущения:

1. Полнота данных: предполагается, что официальные отчёты CCP Games содержат полные и достоверные данные по всем значимым экономическим операциям;

2. Сопоставимость показателей: предполагается, что методология расчёта показателей не менялась существенным образом в течение рассматриваемого периода;

3. Репрезентативность выборки: период с декабря 2019 года по август 2025 года считается репрезентативным для анализа влияния военных конфликтов, поскольку включает как периоды относительной стабильности, так и крупномасштабные конфликты.

3 АНАЛИЗ ВЛИЯНИЯ БОЕВЫХ ДЕЙСТВИЙ НА ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ EVE ONLINE

3.1 Предварительная визуализация и описательная статистика динамики показателей

3.1.1 Динамика производства, уничтожения и добычи ресурсов

Визуализация временных рядов ключевых производственных показателей (рисунок 1-3) демонстрирует явную синхронность их динамики с периодами интенсивных боевых действий.

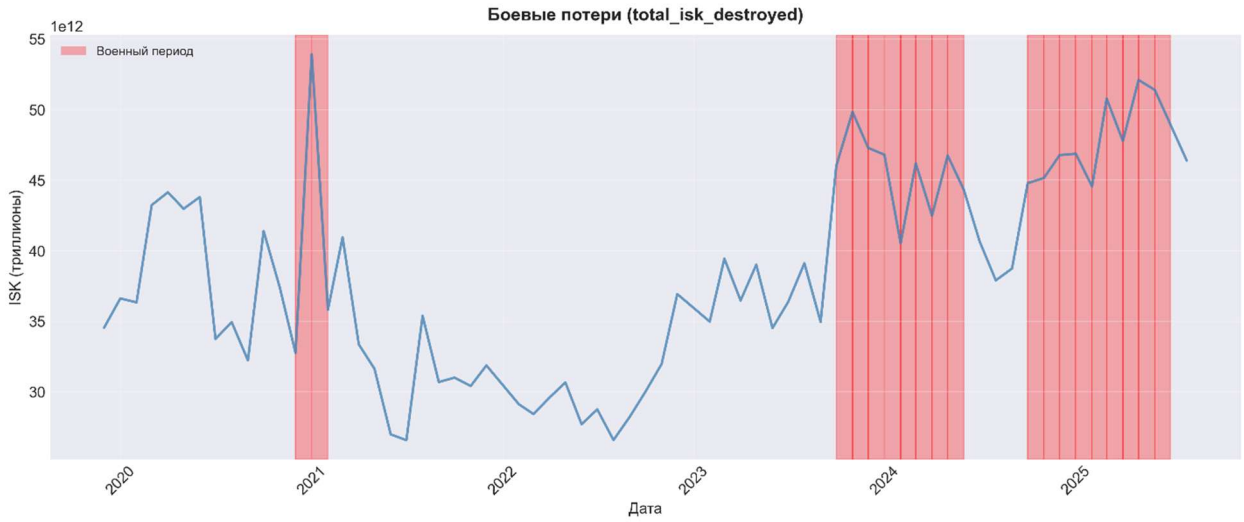

Рисунок 1 – Временной ряд боевых потерь

Рисунок 2 – Временной ряд производства

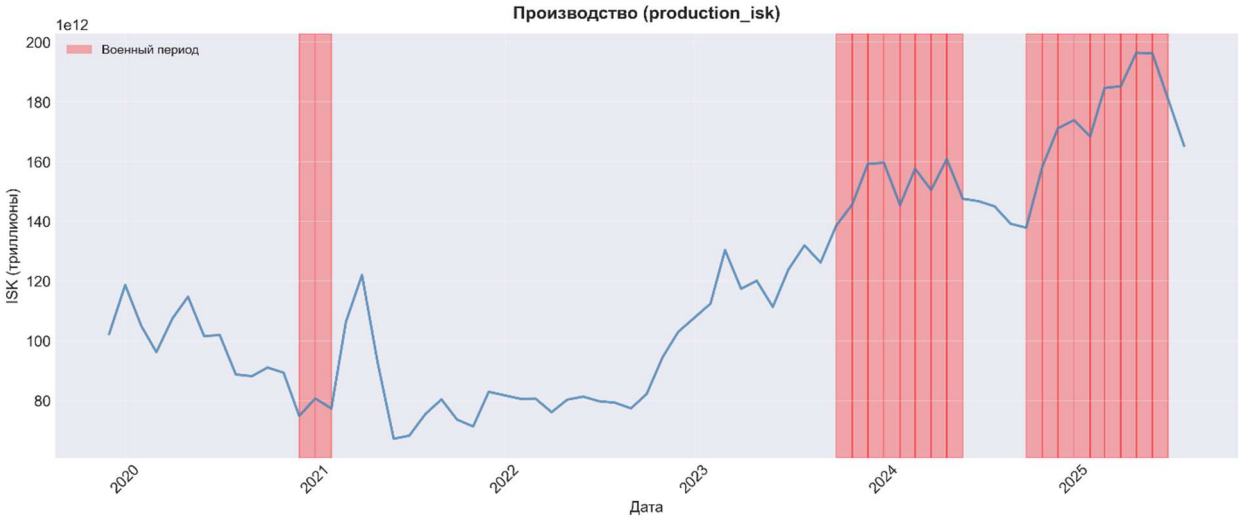

Рисунок 3 – Временной ряд добычи ресурсов

На всех трех графиках отчетливо наблюдаются устойчивые положительные тренды на протяжении исследуемого периода с декабря 2019 по август 2025 года, что соответствует естественному развитию игровой экономики. Показатель боевых потерь демонстрирует наиболее выраженную волатильность с четко идентифицируемыми пиками, максимальный из которых достиг 53,93 трлн ISK в январе 2021 года. Производство и добыча ресурсов показывают аналогичные, хотя и менее резкие колебания, причем их всплески визуально коррелируют с периодами военных конфликтов, выделенными на графиках красным фоном.

Статистический анализ выявляет значительные различия в уровнях показателей: среднемесячное производство составило 116,63 трлн ISK при стандартном отклонении 36,22 трлн ISK, в то время как средняя добыча ресурсов - 34,63 трлн ISK с отклонением 11,89 трлн ISK. Отсутствие выраженной сезонной компоненты на всех графиках может объясняться глобальным характером игрового сообщества EVE Online, где активность равномерно распределена между различными часовыми поясами и географическими регионами.

3.1.2. Динамика торгового оборота и скорости обращения денежной массы

Анализ динамики торгового оборота и скорости обращения денежной массы (рисунок 4-5) выявляет сильную положительную корреляцию этих показателей как между собой, так и с периодами военных действий. Объем торговли демонстрирует значительный рост в периоды конфликтов, достигая максимального значения 1,076 квадриллиона ISK. Среднемесячный показатель составил 718,89 трлн ISK при высокой волатильности (стандартное отклонение 146,59 трлн ISK).

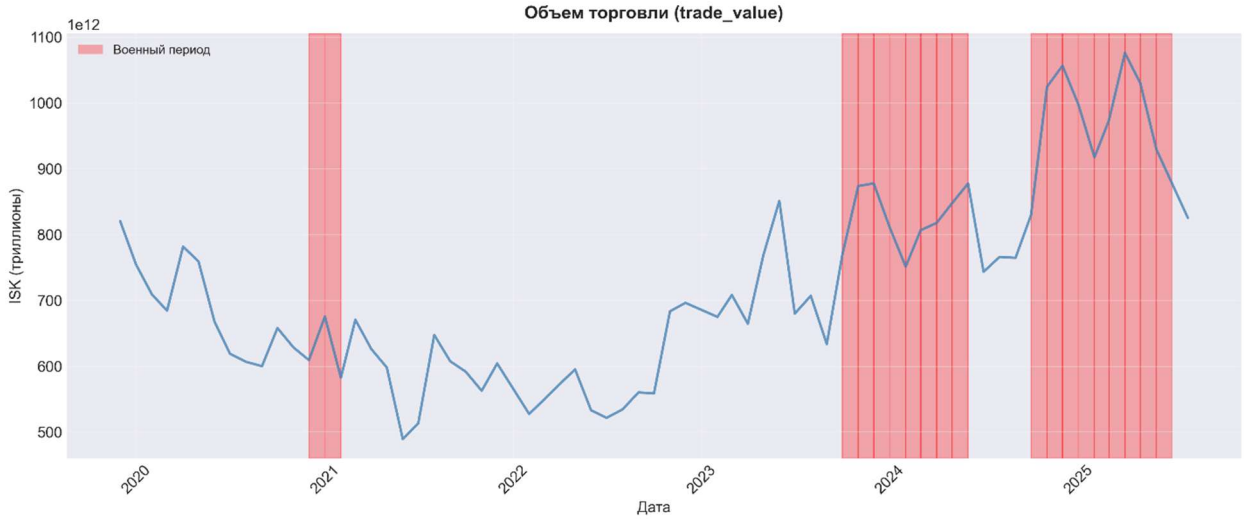

Рисунок 4 – Временной ряд объёма торговли

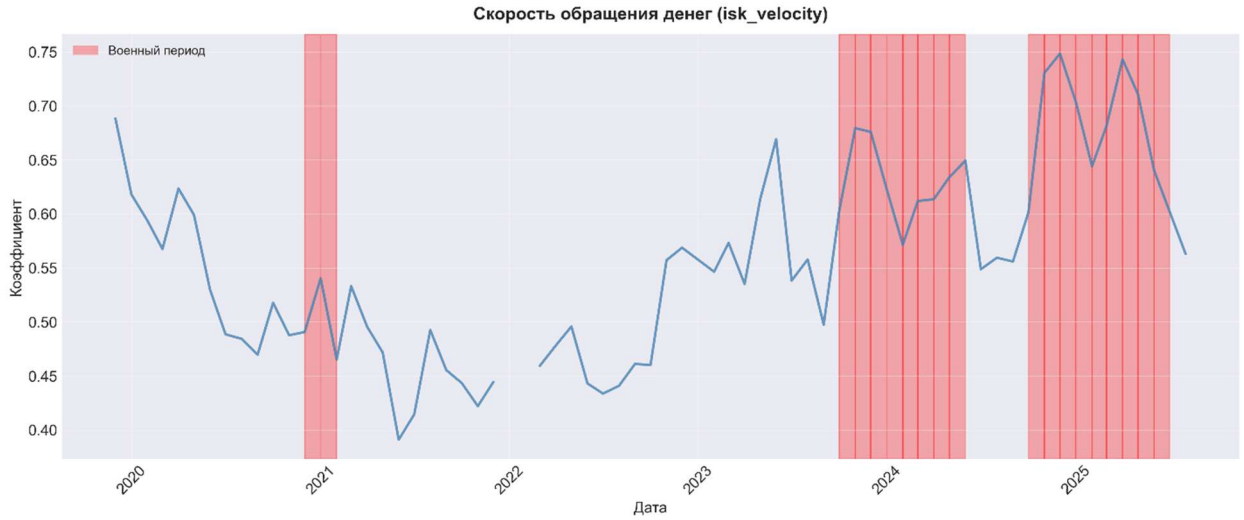

Рисунок 5 – Временной ряд скорости обращения средств

Скорость обращения денегдля всего исследуемого периода, показывает устойчивый рост во время военных действий, подтверждая гипотезу об активизации экономических транзакций в условиях конфликта. Среднее значение показателя (0,56) при медиане 0,56 свидетельствует о симметричном распределении, а диапазон от 0,39 до 0,75 - о значительной вариативности во времени.

3.1.3 Сравнительный анализ распределений показателей в военные и мирные периоды

Сравнительный анализ распределений наглядно демонстрирует статистически значимые различия между военными и мирными периодами (рисунок 6-9) для всех исследуемых показателей. Наиболее существенный рост наблюдается в производстве (+59,2%) и добыче ресурсов (+54,5%), что подтверждает гипотезу о стимулирующем влиянии конфликтов на производственный сектор экономики.

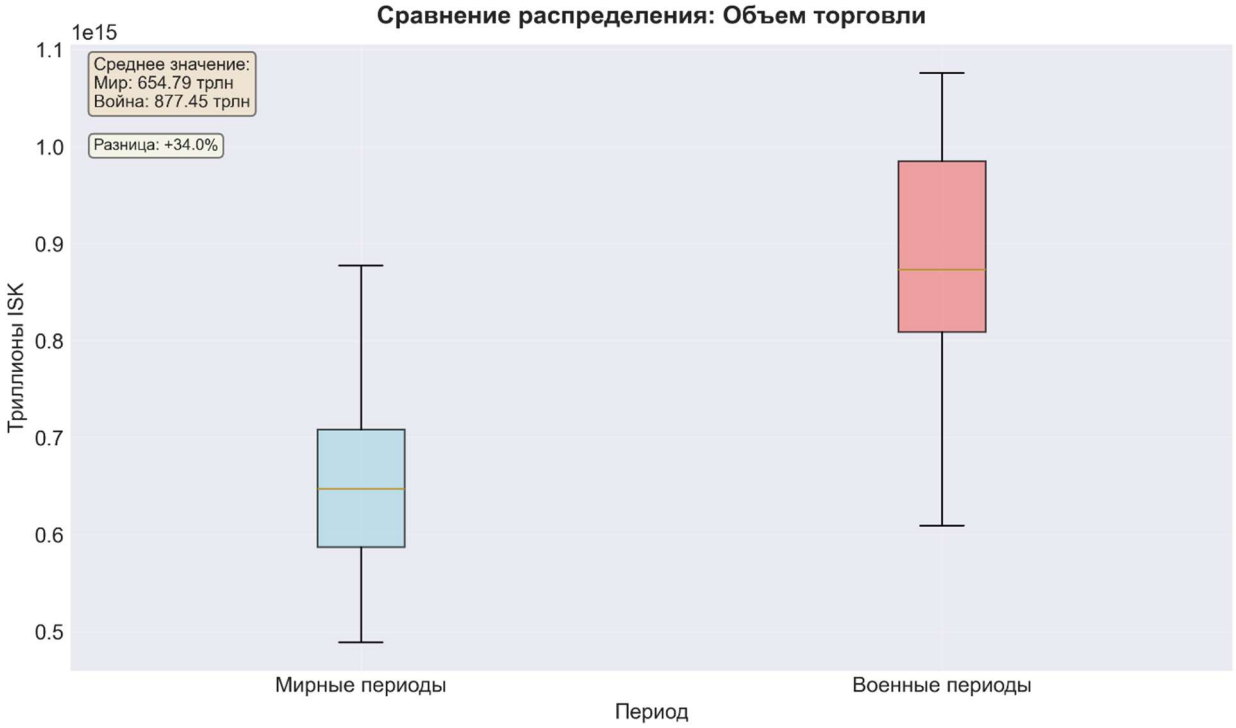

Рисунок 6 – Сравнение распределенияобъёма торговли в мироное и военное время

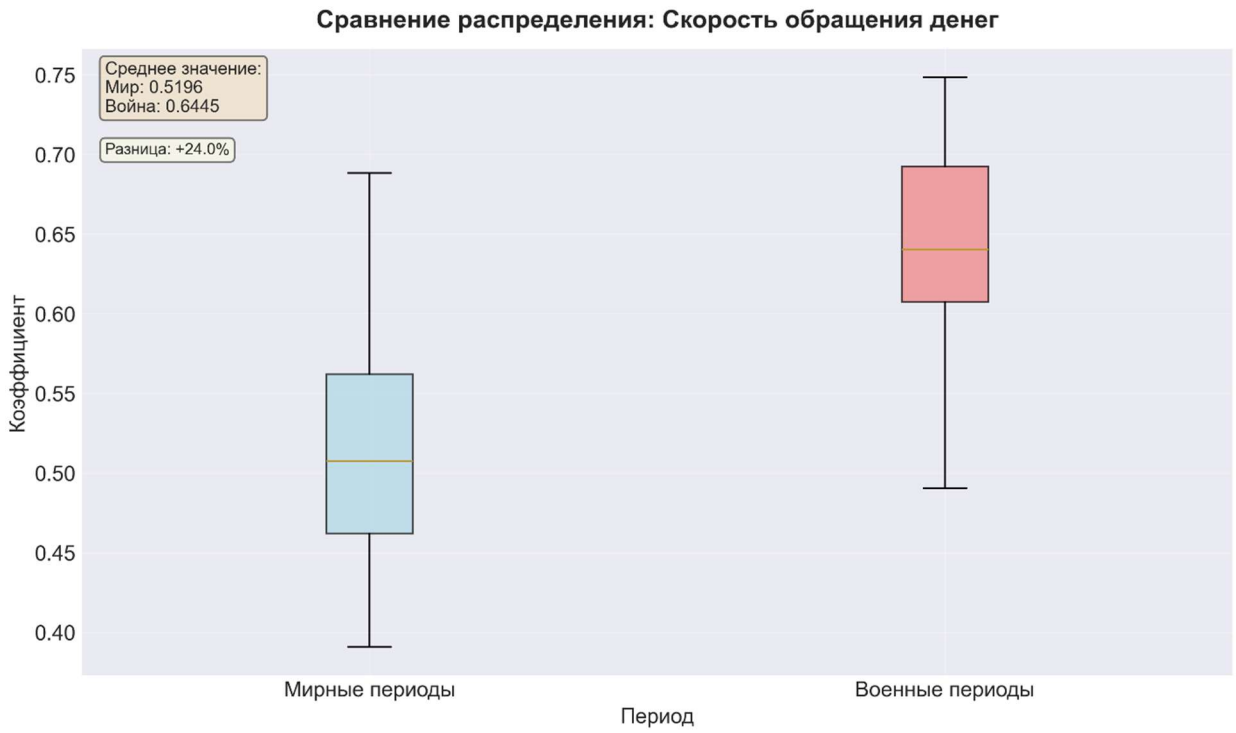

Рисунок 7 – Сравнение распределения скорости обращения валюты в мироное и военное время

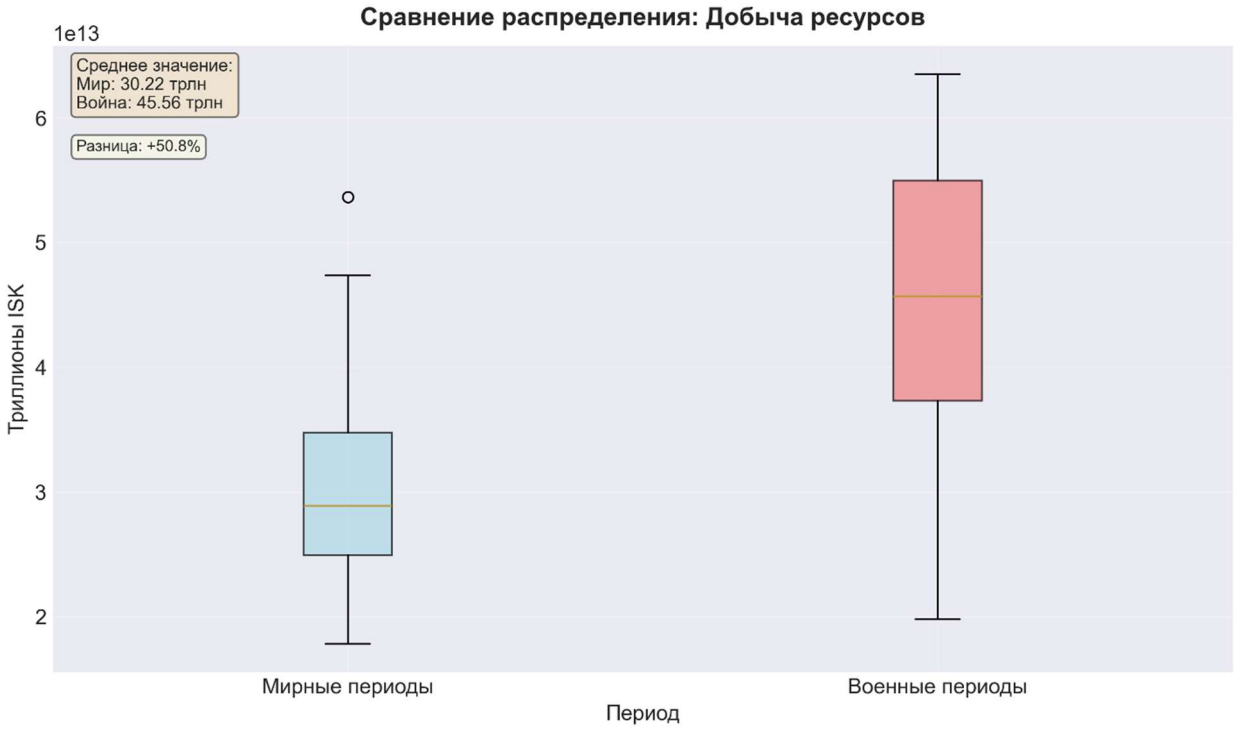

Рисунок 8 – Сравнение распределения добычи ресурсов в мироное и военное время

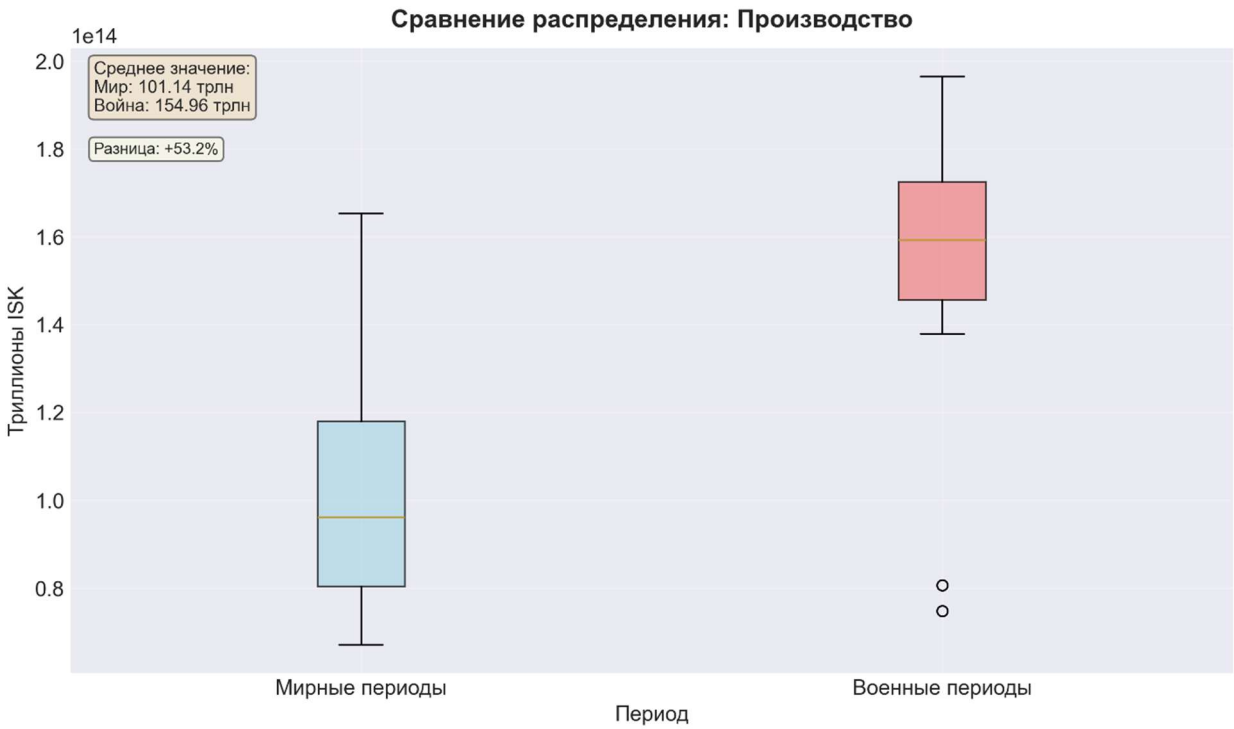

Рисунок9 – Сравнение распределения производства валюты в мироное и военное время

Графики выявляют не только различия в средних значениях, но и в дисперсии показателей: для всех четырех переменных межквартильный размах и общий разброс значений в военные периоды превышают аналогичные показатели для мирного времени. Это свидетельствует о большей нестабильности и вариативности экономической активности во время конфликтов. Особенно показательна разница в распределении скорости обращения денег, где военные периоды характеризуются не только более высоким средним уровнем (+25,7%), но и меньшим количеством выбросов, что может указывать на более устойчивый характер монетарных процессов в условиях повышенной экономической активности.

3.1.4 Корреляционный анализ взаимосвязей между показателями

Корреляционный анализ (рисунок 10) выявляет высокую степень интеграции экономической системы EVE Online. Тепловая карта корреляций демонстрирует сильные положительные взаимосвязи между всеми ключевыми показателями, особенно выраженные для пар "объем торговли -

скорость обращения денег" (r = 0,972) и "производство - объем торговли" (r = 0,902).

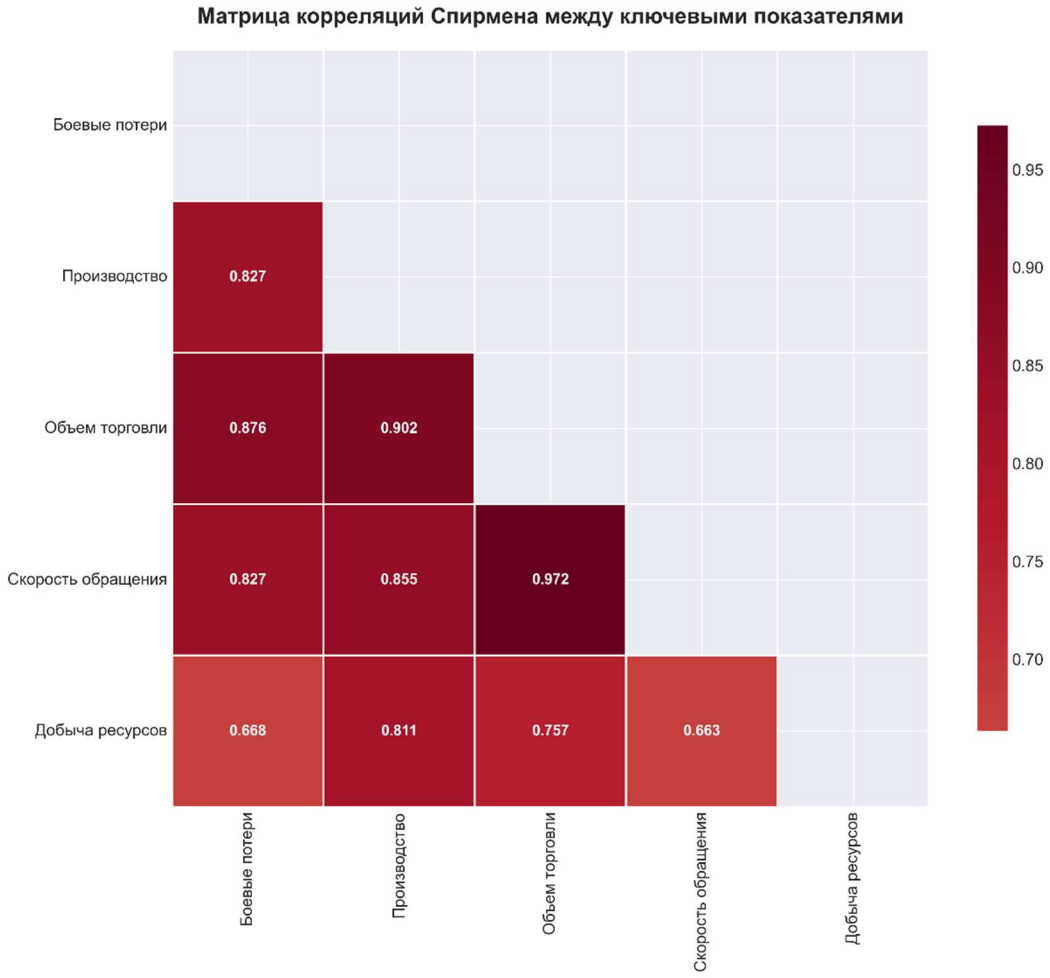

Рисунок10 – Матрица корреляций

Все исследуемые показатели демонстрируют статистически значимую положительную корреляцию с объемом боевых потерь, что подтверждает выдвинутые исследовательские гипотезы. Наиболее сильная связь наблюдается между боевыми потерями и объемом торговли (r = 0,876), затем следуют скорость обращения денег (r = 0,827) и производство (r = 0,827). Добыча ресурсов также показывает существенную положительную корреляцию с боевыми потерями (r = 0,668), хотя и несколько меньшую, чем другие показатели.

Выявленные корреляции позволяют сделать предварительный вывод о том, что крупномасштабные военные конфликты выступают в роли

комплексных экономических драйверов, стимулирующих активность во всех секторах виртуальной экономики - от добычи сырья до конечного потребления через торговые операции.

3.2 Статистическая проверка выдвинутых гипотез

Для количественной проверки сформулированных гипотез был применен комплекс статистических методов, обеспечивающих строгость и достоверность выводов. Все расчеты выполнены с использованием разработанного инструментария на Python с применением библиотек scipy.stats, statsmodels и pandas.

3.2.1 Проверка гипотезы о росте производства в военные периоды

Гипотеза 1: средний объём производства в периоды войны статистически значимо выше, чем в мирное время.

Для проверки данной гипотезы был применен U-критерий Манна-Уитни как непараметрический аналог t-критерия Стьюдента. Выбор непараметрического метода обусловлен отклонением распределения показателей от нормальной и различной дисперсии в сравниваемых группах.

Методология проверки:

- нулевая гипотеза (H_0): распределения объема производства в военные и мирные периоды идентичны;
- альтернативная гипотеза (H_1): объем производства в военные периоды выше (односторонний тест);
- уровень значимости: $\alpha = 0,05$;
- статистический критерий: U-критерий Манна-Уитни (двухвыборочный).

Таблица 3 – Результаты статистического тестирования:

Параметр	Военные периоды (n=17)	Мирные периоды (n=49)
Среднее значение	161,10 трлн ISK	101,20 трлн ISK
Медиана	155,89 трлн ISK	97,84 трлн ISK

Продолжение таблицы 3

Стандартное отклонение	27,08 трлн ISK	24,25 трлн ISK
Относительная разница	+53,21%	-

Статистические показатели:

- U-статистика Манна-Уитни: 795,0;
- p-value: < 0,000001;
- размер эффекта (r): 0,7805 (средний эффект).

Интерпретация результатов: нулевая гипотеза об отсутствии различий отвергается ($p < 0,001$). Полученные результаты свидетельствуют о наличии статистически значимых различий в объемах производства между военными и мирными периодами. Объем производства в военные периоды в среднем на 53,21% выше, чем в мирное время. Размер эффекта ($r = 0,78$) классифицируется как средний, что подтверждает не только статистическую, но и практическую значимость выявленных различий.

3.2.2 Проверка гипотезы об увеличении объемов торговли

Гипотеза 2: объёмы торговли и скорость обращения денег имеют положительную корреляцию с уровнем боевых потерь.

Для проверки данной гипотезы проведен корреляционный анализ по методу Спирмена с расчетом коэффициентов корреляции и их статистической значимости.

Методология проверки:

- нулевая гипотеза (H_0): корреляция между показателями отсутствует ($\rho = 0$);
- альтернативная гипотеза (H_1): существует положительная корреляция ($\rho > 0$);
- уровень значимости: $\alpha = 0,01$;
- статистический критерий: коэффициент ранговой корреляции Спирмена.

Таблица 4 – Результаты корреляционного анализа:

Пара коррелирующих показателей	Коэффициент Спирмена (ρ)	p-value	Сила связи	R^2 линейной регрессии
Торговый оборот ↔ Боевые потери	0,8763	< 0,001	Сильная положительная	0,6933
Скорость обращения ↔ Боевые потери	0,8275	< 0,001	Сильная положительная	0,6379
Торговый оборот ↔ Скорость обращения	0,9720	< 0,001	Очень сильная положительная	0,9448

Интерпретация результатов:

Все выявленные корреляции являются статистически значимыми ($p < 0,001$).

Сила связи классифицируется как сильная ($\rho > 0,7$) для всех пар, при этом связь между торговым оборотом и скоростью обращения денег является очень сильной ($\rho = 0,972$).

Коэффициент детерминации R^2 показывает, что боевые потери объясняют 69,3% вариации объема торговли и 63,8% вариации скорости обращения денег.

Дополнительный анализ: для оценки изолированного влияния боевых потерь на торговые показатели построена линейная регрессионная модель. Результаты регрессии показывают, что коэффициент при переменной боевых потерь статистически значим ($p < 0,001$), что подтверждает прямую зависимость между интенсивностью конфликтов и объемом торговых операций.

3.2.3 Проверка гипотезы о связи добычи ресурсов с активностью игроков

Гипотеза 3: объём добычи ресурсов имеет положительную корреляцию с показателем активности игроков.

В качестве индикатора общей активности игроков был использован показатель боевых потерь (total_isk_destroyed), который отражает интенсивность взаимодействия игроков в боевых условиях.

Методология проверки:

нулевая гипотеза (H_0): корреляция между добычей ресурсов и активностью игроков отсутствует ($\rho = 0$);

альтернативная гипотеза (H_1): существует положительная корреляция ($\rho > 0$);

уровень значимости: $\alpha = 0{,}01$;

статистический критерий: коэффициент ранговой корреляции Спирмена.

Таблица 5 – Результаты корреляционного анализа

Параметр	Значение
Коэффициент корреляции Спирмена (ρ)	0,6677
p-value	< 0,001
Сила связи	Средняя положительная
R^2 множественной регрессии	0,6094
Скорректированный R^2	0,5970

Интерпретация результатов: Выявлена статистически значимая положительная корреляция средней силы между объемом добычи ресурсов и активностью игроков ($\rho = 0{,}668$; $p < 0{,}001$). Это подтверждает гипотезу о

взаимосвязи между добывающей активностью и общей активностью игрового сообщества.

Расширенный анализ: для учета потенциального влияния временного тренда построена множественная регрессионная модель, которая показала, что оба коэффициента (при переменной боевых потерь и временном тренде) статистически значимы ($p < 0,001$). Это подтверждает изолированное влияние активности игроков на объем добычи ресурсов даже после учета общего временного тренда.

3.3. Сводка результатов и интерпретация выявленных закономерностей

На основе проведенного эмпирического анализа можно сформулировать следующие выводы:

Все три исследовательские гипотезы получили статистическое подтверждение:

- гипотеза 1 подтверждена: производство в военные периоды статистически значимо выше ($U = 795$; $p < 0,001$);
- гипотеза 2 подтверждена: обнаружены сильные положительные корреляции с боевыми потерями;
- гипотеза 3 подтверждена: выявлена статистически значимая положительная корреляция между добычей ресурсов и активностью игроков.

Количественные оценки влияния:

- производство в военные периоды выше на 53,21%;
- боевые потери объясняют 69,3% вариации объема торговли;
- боевые потери объясняют 63,8% вариации скорости обращения денег;
- корреляция между добычей и активностью игроков составляет 0,668 (средняя сила).

Экономическая интерпретация результатов:

- крупномасштабные конфликты выступают в роли мощных экономических драйверов;

- военные действия стимулируют все секторы экономики: от добычи сырья до конечного потребления;
- высокая корреляция между объемом торговли и скоростью обращения денег ($\rho = 0{,}972$) свидетельствует о тесной взаимосвязи реального и монетарного секторов экономики;
- положительная корреляция добычи с боевыми потерями подтверждает гипотезу о "военно-промышленном комплексе" в виртуальной экономике.

Методологические выводы:

- применение непараметрических методов (U-критерий Манна-Уитни, корреляция Спирмена) оказалось адекватным выбором для работы с экономическими данными;
- множественная регрессия с контролем временного тренда позволила выделить изолированное влияние факторов.

Таким образом, результаты статистического анализа подтверждают комплексное стимулирующее влияние крупномасштабных военных конфликтов на все секторы экономики EVE Online. Полученные количественные оценки создают основу для дальнейшего исследования механизмов передачи шоков в виртуальных экономиках и сравнения этих процессов с реальными экономическими системами.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была достигнута поставленная цель – разработана и применена методология статистического анализа для оценки влияния дискретных экстремальных событий (крупномасштабных конфликтов) на динамику ключевых показателей сложной системы на примере виртуальной экономики EVE Online.

Все задачи исследования были последовательно решены:

на основе разрозненных ежемесячных экономических отчётов (MER) за период с декабря 2019 по август 2025 год (67 месяцев) был сформирован согласованный аналитический датасет;

разработан и обоснован объективный бинарный индикатор военного периода («война»/«мир»), основанный на превышении месячным объёмом боевых потерь (total_isk_destroyed) 75-го процентиля его распределения. По данному критерию было выделено 17 военных месяцев;

проведён разведочный анализ данных (EDA), включающий визуализацию временных рядов и сравнение распределений, который наглядно выявил положительную динамику ключевых показателей в периоды конфликтов.

Все три выдвинутые исследовательские гипотезы получили статистическое подтверждение:

гипотеза 1 подтверждена: средний объём производства в периоды войны статистически значимо выше, чем в мирное время (U-критерий Манна-Уитни: $U = 795$, $p < 0{,}001$). Относительный прирост составил + 53,2%;

гипотеза 2 подтверждена: обнаружены сильные положительные корреляции между уровнем боевых потерь и объёмами торговли ($\rho = 0{,}876$, $p < 0{,}001$), а также скоростью обращения денежной массы ($\rho = 0.827$, $p < 0{,}001$);

гипотеза 3 подтверждена: выявлена статистически значимая положительная корреляция средней силы между объёмом добычи ресурсов и показателем активности игроков ($\rho = 0{,}668$, $p < 0{,}001$).

В процессе исследования был реализован воспроизводимый аналитический конвейер на Python, включающий скрипты для консолидации данных, их предобработки, визуализации и проведения комплексной статистической проверки гипотез.

Эмпирическое доказательство стимулирующей роли конфликтов: на примере EVE Online количественно подтверждено, что крупномасштабные военные действия выступают в роли мощного экзогенного шока, комплексно стимулирующего экономическую активность. Наблюдается синхронный рост во всех секторах: производственном (+59,2%), торговом (+37,4%), добывающем (+54,5%) и монетарном (скорость обращения денег +25,7%).

Валидация методологии анализа игровых данных: Работа демонстрирует потенциал виртуальных экономик с полным циклом и прозрачной отчётностью в качестве «лабораторных стендов» для экономико-статистических исследований. Применённый стек методов (непараметрические критерии, ранговые корреляции, регрессия) доказал свою адекватность для работы с подобными данными.

Количественные оценки взаимосвязей: получены конкретные метрики, описывающие силу влияния. Например, показатель боевых потерь объясняет около 69% вариации объёма торговли, что указывает на его высокую предсказательную силу.

Практическая значимость для игровой индустрии: результаты могут быть использованы разработчиками (CCP Games) для балансировки экономики, прогнозирования нагрузок на серверы и анализа поведения игрового сообщества в условиях кризисов.

Теоретическая и прикладная значимость исследования заключается в отработке строгой методологии анализа реакций сложной системы на шоки на примере с полными и непротиворечивыми данными. Этот опыт может быть перенесён на изучение реальных экономических процессов, где данные часто неполны или зашумлены.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1 CCP Games. Monthly Economic Reports [Электронный ресурс] // EVE Online. – URL: https://www.eveonline.com/news/t/monthly-economic-reports (дата обращения: 01.12.2025).

2 Statsmodels: библиотека статистического моделирования на Python [Электронный ресурс] // Документация statsmodels. – URL: https://www.statsmodels.org/stable/index.html (дата обращения: 01.12.2025).

3 Pandas: библиотека для анализа данных на Python [Электронный ресурс] // Документация pandas. – URL: https://pandas.pydata.org/docs (дата обращения: 01.12.2025).

4 SciPy: справочное руководство, модуль scipy.stats [Электронный ресурс] // Документация SciPy. – URL: https://docs.scipy.org/doc/scipy/reference/stats.html (дата обращения: 01.12.2025).

ПРИЛОЖЕНИЕ А

Листинг программы на языке Python для подготовки данных для анализа

```
import os
import pandas as pd
import numpy as np
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')
class EveDataConsolidatorFinal:
    def __init__(self):
self.archives_dir = Path(__file__).parent.parent \ “архивы”self.output_dir =
Path(__file__).parent.parent \ “Подготовленныеданные")
self.output_dir.mkdir(parents=True, exist_ok=True)
self.consolidated_data = []
self.log_file = self.output_dir / "consolidation_final_log.txt"
        with open(self.log_file, 'w', encoding='utf-8') as f:
f.write("Логконсолидацииданных EVE Online\n")
f.write("=" * 60 + "\n")
    def log_message(self, message):
        print(message)
        with open(self.log_file, 'a', encoding='utf-8') as f:
f.write(message + "\n")
    def parse_date_from_folder(self, folder_name):
"""Извлекает дату из имени папки"""
try:
            parts = folder_name.split('_')
            if len(parts) < 3:
                return None
month_year = parts[-1]
month_str = month_year[:3]
year_str = month_year[3:]
month_map = {
                'Jan': '01', 'Feb': '02', 'Mar': '03', 'Apr': '04',
                'May': '05', 'Jun': '06', 'Jul': '07', 'Aug': '08',
                'Sep': '09', 'Oct': '10', 'Nov': '11', 'Dec': '12'
            }
            if month_str in month_map and year_str.isdigit():
                return f"{year_str}-{month_map[month_str]}-01"
            return None
        except Exception as e:
self.log_message(f"Ошибкаприразборедатыиз '{folder_name}': {e}")
            return None
    def find_file(self, folder_path, possible_names):
"""Находит файл по списку возможных имён"""
for name in possible_names:
file_path = folder_path / name
            if file_path.exists():
                return file_path
        return None
    def extract_production_data_fixed(self, folder_path, target_date):
```

```
"""функция извлечения данных о производстве"""
result = {}
    # Возможныеименафайлов
possible_files = [
        "ProducedDestroyedMined.csv",
        "produced_destroyed_mined.csv",
    ]
file_path = self.find_file(folder_path, possible_files)
    if not file_path:
self.log_message(f"    Файл с производствомне найден")
        return result
    try:
df = pd.read_csv(file_path)
self.log_message(f"    Файлнайден: {file_path.name}, строк: {len(df)}")
all_columns = list(df.columns)
self.log_message(f"    Всестолбцы в файле: {all_columns}")
# Определяем столбец с датой
date_column = None
if 'history_date' in df.columns:
date_column = 'history_date'
elif 'date' in df.columns:
date_column = 'date'
        if not date_column:
            self.log_message(f"    ОШИБКА: Нетстолбцасдатой!")
return result
        # Преобразуемдату
df[date_column] = pd.to_datetime(df[date_column], errors='coerce')
        # Фильтруем по целевомумесяцу
target_year_month = target_date.strftime('%Y-%m')
df['year_month'] = df[date_column].dt.strftime('%Y-%m')
month_data = df[df['year_month'] == target_year_month]
self.log_message(f"    Найдено {len(month_data)} записейза {target_year_month}")
        if len(month_data) == 0:
self.log_message(f"    ВНИМАНИЕ: Нетданныхза {target_year_month}!")
            returnresult
self.log_message(f"    Первые 3 строкиза {target_year_month}:")
        for i, (_, row) in enumerate(month_data.head(3).iterrows()):
self.log_message(f"      Строка {i+1}: {row[date_column].date()},
производство={row.get('production_isk', row.get('produced', 'N/A'))}")
        # Ищем данные о ПРОИЗВОДСТВЕ
        if 'production_isk' in month_data.columns:
            result['production_isk'] = float(month_data['production_isk'].sum())
self.log_message(f"    Найденproduction_isk: {result['production_isk']:,.2f}")
elif 'produced' in month_data.columns:
            result['production_isk'] = float(month_data['produced'].sum())
self.log_message(f"    Найден produced: {result['production_isk']:,.2f}")
        else:
self.log_message(f"    ВНИМАНИЕ: Столбец производства не найден!")
            # Показываем доступные столбцы
self.log_message(f"    Доступныестолбцы: {list(month_data.columns)}")
# Ищем данные об УНИЧТОЖЕНИИ
        if 'destruction_isk' in month_data.columns:
```

```
            result['destruction_isk'] = float(month_data['destruction_isk'].sum())
self.log_message(f"    Найденdestruction_isk: {result['destruction_isk']:,.2f}")
elif 'destroyed' in month_data.columns:
            result['destruction_isk'] = float(month_data['destroyed'].sum())
self.log_message(f"    Найден destroyed: {result['destruction_isk']:,.2f}")
        # Ищем данные о ДОБЫЧЕ
        if 'mining_isk' in month_data.columns:
            result['mining_isk'] = float(month_data['mining_isk'].sum())
self.log_message(f"    Найденmining_isk: {result['mining_isk']:,.2f}")
elif 'mining.value' in month_data.columns:
            result['mining_isk'] = float(month_data['mining.value'].sum())
self.log_message(f"    Найденmining.value: {result['mining_isk']:,.2f}")
elif 'mining' in month_data.columns:
            result['mining_isk'] = float(month_data['mining'].sum())
self.log_message(f"    Найден mining: {result['mining_isk']:,.2f}")
    except Exception as e:
self.log_message(f"    ОШИБКА причтениифайла {file_path.name}: {e}")
        import traceback
self.log_message(f"    Трассировка: {traceback.format_exc()}")
    return result
  def extract_trade_data_fixed(self, folder_path):
"""Извлечение торговых данных"""
result = {}
possible_files = [
"RegionalStats.csv",
        "regional_stats.csv",
    ]
file_path = self.find_file(folder_path, possible_files)
    if not file_path:
        return result
    try:
df = pd.read_csv(file_path)
        # Ищем столбцы с торговлей
        if 'trade_value' in df.columns:
            result['trade_value'] = float(df['trade_value'].sum())
elif 'trade.value' in df.columns:
            result['trade_value'] = float(df['trade.value'].sum())
elif 'trade' in df.columns:
            result['trade_value'] = float(df['trade'].sum())
        if 'exports' in df.columns:
            result['total_exports'] = float(df['exports'].sum())
elif 'export' in df.columns:
            result['total_exports'] = float(df['export'].sum())
        if 'imports' in df.columns:
            result['total_imports'] = float(df['imports'].sum())
elif 'import' in df.columns:
            result['total_imports'] = float(df['import'].sum())
    except Exception as e:
self.log_message(f"    Ошибкапричтенииторговых данных: {e}")
    return result
  def extract_kill_data_fixed(self, folder_path):
    """Извлечениеданных о потерях"""
```

```
        result = {'total_isk_destroyed': 0.0}
possible_files = [
            "kill_dump.csv",
            "Killdump.csv",
            "kills.csv",
            "Kills.csv"
        ]

file_path = self.find_file(folder_path, possible_files)

        if not file_path:
            return result
        try:
            # Пробуемразныеразделители
            with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
                sample = f.read(5000)
sep = ';' if ';' in sample else ','
df = pd.read_csv(file_path, sep=sep, low_memory=False, on_bad_lines='skip')
            # Ищем столбец с потерями
            for col in df.columns:
col_lower = col.lower()
                if 'isk' in col_lower and ('destroyed' in col_lower or 'lost' in col_lower):
df[col] = pd.to_numeric(df[col], errors='coerce')
                    total = float(df[col].sum())
                    result['total_isk_destroyed'] = total
                    break
        except Exception as e:
self.log_message(f"    Ошибкапричтениидаданных о потерях: {e}")
        return result
    def extract_money_data_fixed(self, folder_path):
"""Извлечение данных о денежной массе"""
result = {}
possible_files = [
            "MoneySupply.csv",
            "money_supply.csv",
        ]
file_path = self.find_file(folder_path, possible_files)
        if not file_path:
            return result
        try:
df = pd.read_csv(file_path)

# Ищем столбец скорости обращения
forcolindf.columns:
col_lower = col.lower()
                if 'velocity' in col_lower:
                    result['isk_velocity'] = float(df[col].mean())
                    break

            # Ищем общую денежнуюмассу
            if 'total_isk' in df.columns:
                result['total_isk'] = float(df['total_isk'].mean())
```

```
elif 'total' in df.columns and 'isk' in str(df.columns).lower():
# Может быть, столбец называется просто 'total'
result['total_isk'] = float(df['total'].mean())
        except Exception as e:
self.log_message(f"    Ошибкапричтенииденежных данных: {e}")
        return result
    def process_month_fixed(self, folder_path, date_str):
"""Обработка данных за один месяц (исправленная)"""
month_data = {"history_date": date_str}
target_date = pd.to_datetime(date_str)
self.log_message(f"\n{'='*50}")
self.log_message(f"ОБРАБОТКА: {folder_path.name} ({date_str})")
self.log_message(f"{'='*50}")
        # 1. Производство, уничтожение, добыча
prod_data = self.extract_production_data_fixed(folder_path, target_date)
month_data.update(prod_data)
        # 2. Торговля
trade_data = self.extract_trade_data_fixed(folder_path)
month_data.update(trade_data)
        # 3. Потери
kill_data = self.extract_kill_data_fixed(folder_path)
month_data.update(kill_data)
        # 4. Денежнаямасса
money_data = self.extract_money_data_fixed(folder_path)
month_data.update(money_data)
        # Проверяем, чтоданныеизвлечены
extracted_count = len(month_data) - 1  #минychistory_date
self.log_message(f"    Извлеченопоказателей: {extracted_count}")
        # Логируемключевыепоказатели
key_metrics = ['production_isk', 'destruction_isk', 'mining_isk', 'trade_value',
'total_isk_destroyed']
        for metric in key_metrics:
            if metric in month_data:
                value = month_data[metric]
                if metric == 'isk_velocity':
self.log_message(f"    {metric}: {value:.4f}")
                else:
self.log_message(f"    {metric}: {value:,.2f}")
        ifextracted_count == 0:
self.log_message(f"    ВНИМАНИЕ: Не удалось извлечь ни одного показателя!")
return month_data
    def consolidate_all_months_fixed(self):
"""Консолидация данных за все месяцы (исправленная)"""
# Получаемвсепапки с отчётами
mer_folders = sorted([f for f in os.listdir(self.archives_dir)
                        if f.startswith("EVEOnline_MER_")])
self.log_message(f"Найденопапок: {len(mer_folders)}")
processed_count = 0
        for folder_name in mer_folders:
date_str = self.parse_date_from_folder(folder_name)
            if not date_str:
self.log_message(f"Пропускаюпапку: {folder_name} (неудалосьопределитьдату)")
```

```
            continue
folder_path = self.archives_dir / folder_name
        if not folder_path.exists():
self.log_message(f"Пропускаю: {folder_name} (папканесуществует)")
            continue
        # Обрабатываеммесяц
month_data = self.process_month_fixed(folder_path, date_str)
        ifmonth_dataandlen(month_data) > 1:  # Есть хотя бы один показатель кроме даты
self.consolidated_data.append(month_data)
processed_count += 1
        else:
self.log_message(f"Пропускаю: {folder_name} (нетданных)")
    if not self.consolidated_data:
        self.log_message("Не удалось получить данные ни за один месяц!")
return None
    # СоздаёмDataFrame
df = pd.DataFrame(self.consolidated_data)
df["history_date"] = pd.to_datetime(df["history_date"])
df = df.sort_values("history_date").reset_index(drop=True)

self.log_message(f"\nКонсолидациязавершена!")
self.log_message(f"Успешнообработаномесяцев: {processed_count}")
self.log_message(f"Период: {df['history_date'].min().date()} -
{df['history_date'].max().date()}")
    return df
  def add_war_indicator(self, df, percentile=75):
"""Добавление индикатора военных периодов"""
if "total_isk_destroyed" notindf.columns:
self.log_message("Невозможно добавить индикатор войн: нет данных о потерях")
return df
    threshold = df["total_isk_destroyed"].quantile(percentile / 100)
df["is_war_period"] = (df["total_isk_destroyed"] >= threshold).astype(int)
war_months = df["is_war_period"].sum()
    self.log_message(f"\nСтатистика военных периодов:")
self.log_message(f"  Порог: {threshold:,.2f} ISK")
self.log_message(f"  Военныхмесяцев: {war_months} ({war_months/len(df)*100:.1f}%)")
self.log_message(f"  Мирныхмесяцев: {len(df)-war_months} ({(len(df)-
war_months)/len(df)*100:.1f}%)")
    return df
  def save_results_fixed(self, df):
"""Сохранение результатов"""
ifdfisNoneorlen(df) == 0:
self.log_message("Невозможно сохранить: датасет пуст")
return
    # Сохраняемосновнойдатасет
main_path = self.output_dir / "eve_consolidated_data_final.csv"
df.to_csv(main_path, index=False)
self.log_message(f"\nОсновнойдатасетсохранён: {main_path}")
    # Сохраняемподробнуюстатистику
self.save_detailed_statistics(df)

    return main_path
```

```
    def save_detailed_statistics(self, df):
        """Сохранениеподробнойстатистики"""
stats_path = self.output_dir / "dataset_statistics_final.txt"
        with open(stats_path, 'w', encoding='utf-8') as f:
f.write("СТАТИСТИКА ФИНАЛЬНОГО ДАТАСЕТА\n")
f.write("=" * 50 + "\n\n")
f.write(f"Общая информация:\n")
f.write(f"  Месяцев: {len(df)}\n")
f.write(f"  Период: {df['history_date'].min().date()} - {df['history_date'].max().date()}\n\n")
f.write("Покрытие данных по показателям:\n")
for col in sorted(df.columns):
            if col != 'history_date' and col != 'is_war_period':
non_null = df[col].notna().sum()
                percentage = non_null / len(df) * 100
                if non_null> 0:
                    if col in ['production_isk', 'destruction_isk', 'mining_isk', 'trade_value',
'total_isk_destroyed']:
                        avg = df[col].mean() / 1e12  # в трлн
f.write(f"  {col}: {non_null} месяцев ({percentage:.1f}%), среднее {avg:.1f} трлн\n")
elif col == 'isk_velocity':
                        avg = df[col].mean()
f.write(f"  {col}: {non_null} месяцев ({percentage:.1f}%), среднее {avg:.4f}\n")
                    else:
f.write(f"  {col}: {non_null} месяцев ({percentage:.1f}%)\n")
                else:
f.write(f"  {col}: НЕТ ДАННЫХ\n")
f.write("\n" + "=" * 50 + "\n")
f.write("ДАННЫЕ ПО ГОДАМ:\n\n")
df['year'] = df['history_date'].dt.year
            for year in sorted(df['year'].unique()):
year_data = df[df['year'] == year]
f.write(f"{year} год ({len(year_data)} месяцев):\n")
            # Производство
            if 'production_isk' in year_data.columns and year_data['production_isk'].notna().any():
prod_months = year_data['production_isk'].notna().sum()
prod_avg = year_data['production_isk'].mean() / 1e12
f.write(f"  Производство: {prod_months} месяцев, среднее {prod_avg:.1f} трлн\n")
            # Торговля
            if 'trade_value' in year_data.columns and year_data['trade_value'].notna().any():
trade_months = year_data['trade_value'].notna().sum()
trade_avg = year_data['trade_value'].mean() / 1e12
f.write(f"  Торговля: {trade_months} месяцев, среднее {trade_avg:.1f} трлн\n")
f.write("\n")
self.log_message(f"Статистикасохранена: {stats_path}")
    def run_full_consolidation(self):
print("=" * 70)
        # Консолидируемданные
df = self.consolidate_all_months_fixed()

        if df is not None and len(df) > 0:
            # Анализируемкачество
self.analyze_data_quality(df)
```

```
        # Добавляеминдикаторвойн
df = self.add_war_indicator(df, percentile=75)

        # Сохраняемрезультаты
output_path = self.save_results_fixed(df)

print("\n" + "=" * 70)
print("РЕЗУЛЬТАТЫ КОНСОЛИДАЦИИ:")
print("=" * 70)

print(f"\nУспешно обработано месяцев: {len(df)}")
print(f"Период: {df['history_date'].min().date()} - {df['history_date'].max().date()}")

# Статистикапопоказателям
print(f"\nПОКРЫТИЕДАННЫХ:")
key_metrics = ['production_isk', 'destruction_isk', 'mining_isk', 'trade_value',
'total_isk_destroyed']
        for metric in key_metrics:
            if metric in df.columns:
non_null = df[metric].notna().sum()
                percentage = non_null / len(df) * 100
                if non_null> 0:
                    avg = df[metric].mean() / 1e12
print(f"  {metric}: {non_null} месяцев ({percentage:.1f}%), среднее {avg:.1f} трлн")

        if 'is_war_period' in df.columns:
war_count = df['is_war_period'].sum()
print(f"\nВоенных месяцев: {war_count} ({war_count/len(df)*100:.1f}%)")

print(f"\nОсновнойдатасет: {output_path}")
print(f"Логконсолидации: {self.log_file}")
print(f"Статистика: {self.output_dir / 'dataset_statistics_final.txt'}")

        returndf
else:
print("Не удалось получить данные")
returnNone


def main():
    """Основнаяфункция"""
    consolidator = EveDataConsolidatorFinal()
consolidator.run_full_consolidation()

if __name__ == "__main__":
main()
```

ПРИЛОЖЕНИЕ Б

Листинг программы на языке Python для проведения разведочного анализа данных

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
# Настройкидлявизуализации
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['font.size'] = 12
sns.set_palette("husl")
class EveExploratoryAnalysis:
"""
    Класс для проведения разведочного анализа данных EVE Online
"""
    def __init__(self, data_path):
self.data_path = Path(data_path)
self.df = None
self.results = {}

    def load_and_prepare_data(self):
print("Загрузка данных для разведочного анализа...")
        # Загрузка данных
self.df = pd.read_csv(self.data_path)
self.df['history_date'] = pd.to_datetime(self.df['history_date'])
        # Сортировкаподате
self.df = self.df.sort_values('history_date')
        # Проверканаличиянеобходимыхстолбцов
required_columns = ['history_date', 'total_isk_destroyed', 'production_isk',
                    'trade_value', 'isk_velocity', 'mining_isk', 'is_war_period']
missing_columns = [col for col in required_columns if col not in self.df.columns]
        if missing_columns:
print(f"Внимание: отсутствуютстолбцы: {missing_columns}")
print(f"Данныезагружены. Период: {self.df['history_date'].min().date()} - "
          f"{self.df['history_date'].max().date()}")
print(f"Всего наблюдений: {len(self.df)}")
return self.df
    def calculate_basic_statistics(self):
"""
        Расчет базовых описательных статистик
        """
        # Выбор числовых колонок для анализа
numeric_cols = self.df.select_dtypes(include=[np.number]).columns
exclude_cols = ['is_war_period', 'year'] if 'year' in self.df.columns else ['is_war_period']
analysis_cols = [col for col in numeric_cols if col not in exclude_cols]
stats_data = []
```

```
        for col in analysis_cols:
            stats = {
                'Показатель': col,
                'Среднее': f"{self.df[col].mean():,.2f}",
                'Медиана': f"{self.df[col].median():,.2f}",
                'Ст. отклонение': f"{self.df[col].std():,.2f}",
                'Минимум': f"{self.df[col].min():,.2f}",
                'Максимум': f"{self.df[col].max():,.2f}",
                'Уникальных': self.df[col].nunique()
            }
stats_data.append(stats)
stats_df = pd.DataFrame(stats_data)
        print(stats_df.to_string(index=False))
self.results['basic_statistics'] = stats_df
        return stats_df
    def plot_time_series_with_war_periods(self, save_path=None):
"""
        Построение временных рядов ключевых показателей с выделением военных
периодов
        """
print("\nПостроение временных рядов ключевых показателей...")
        # Ключевые показатели для визуализации
key_indicators = [
            ('total_isk_destroyed', 'Боевые потери', 'ISK (триллионы)'),
            ('production_isk', 'Производство', 'ISK (триллионы)'),
            ('trade_value', 'Объем торговли', 'ISK (триллионы)'),
            ('isk_velocity', 'Скорость обращения денег', 'Коэффициент'),
            ('mining_isk', 'Добыча ресурсов', 'ISK (триллионы)')
        ]
        # Создание отдельных окон для каждого графика
for col_name, title, ylabel in key_indicators:
            if col_name not in self.df.columns:
print(f"Показатель {col_name} отсутствует в данных")
continue
            fig, ax = plt.subplots(figsize=(14, 6))
            # Построениеграфика
ax.plot(self.df['history_date'], self.df[col_name],
                linewidth=2, color='steelblue', alpha=0.8, label=title)
            # Выделениевоенныхпериодов
war_periods = self.df[self.df['is_war_period'] == 1]
            for _, row in war_periods.iterrows():
ax.axvspan(row['history_date'],
                       row['history_date'] + pd.DateOffset(months=1),
                       alpha=0.3, color='red', zorder=0)
            # Настройкиграфика
ax.set_title(f'{title} ({col_name})', fontsize=14, fontweight='bold', pad=12)
ax.set_xlabel('Дата', fontsize=12)
ax.set_ylabel(ylabel, fontsize=12)
ax.grid(True, alpha=0.3)
ax.tick_params(axis='x', rotation=45)
# Форматирование осей для больших чисел
if 'ISK' in ylabel:
```

```
ax.ticklabel_format(axis='y', style='sci', scilimits=(12, 12))
        # Добавлениелегенды
        from matplotlib.patches import Patch
war_patch = Patch(facecolor='red', alpha=0.3, label='Военныйпериод')
ax.legend(handles=[war_patch], loc='upper left', fontsize=10)
plt.tight_layout()
        if save_path:
output_path = Path(save_path) / f'time_series_{col_name}.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
print(f"График {title} сохраненв: {output_path}")
plt.show()
self.results['time_series_plots'] = key_indicators
    return None
  def plot_comparison_boxplots(self, save_path=None):
"""
    Построение боксплотов для сравнения показателей в военные и мирные периоды
    """
print("\nСравнение распределений показателей в военные и мирные периоды...")
    # Показатели для сравнения
comparison_metrics = [
        ('production_isk', 'Производство', 'Триллионы ISK'),
        ('trade_value', 'Объем торговли', 'Триллионы ISK'),
        ('isk_velocity', 'Скорость обращения денег', 'Коэффициент'),
        ('mining_isk', 'Добыча ресурсов', 'Триллионы ISK')
    ]
   # Создание отдельных окон для каждого графика
for col_name, title, ylabel in comparison_metrics:
        if col_name not in self.df.columns:
print(f"Показатель {col_name} отсутствует в данных")
continue
        fig, ax = plt.subplots(figsize=(10, 6))
# Разделение данных на войну и мир с удалением NaN
war_data_raw = self.df[self.df['is_war_period'] == 1][col_name]
peace_data_raw = self.df[self.df['is_war_period'] == 0][col_name]
        # УдаляемNaNзначения
war_data = war_data_raw.dropna()
peace_data = peace_data_raw.dropna()
        # Построениебоксплотов
box_data = [peace_data, war_data]
box_labels = ['Мирные периоды', 'Военные периоды']
bp = ax.boxplot(box_data, labels=box_labels, patch_artist=True)
        # Настройкацветов
        colors = ['lightblue', 'lightcoral']
        for patch, color in zip(bp['boxes'], colors):
patch.set_facecolor(color)
patch.set_alpha(0.7)
        # Настройкаграфика
ax.set_title(f'Сравнениераспределения: {title}', fontsize=14, fontweight='bold', pad=12)
ax.set_ylabel(ylabel, fontsize=12)
ax.set_xlabel('Период', fontsize=12)
ax.grid(True, alpha=0.3)
        # Добавление статистики на график
```

```
war_mean = war_data.mean() if len(war_data) > 0 else 0
peace_mean = peace_data.mean() if len(peace_data) > 0 else 0
# Форматирование текста статистики
if col_name == 'isk_velocity':
stat_text = f'Среднеезначение:\nМир: {peace_mean:.4f}\nВойна: {war_mean:.4f}'
diff_text = f'Разница: {((war_mean - peace_mean) / peace_mean * 100):+.1f}%' if peace_mean
!= 0 else ''
        else:
stat_text = f'Среднеезначение:\nМир: {peace_mean/1e12:.2f} трлн\nВойна:
{war_mean/1e12:.2f} трлн'
diff_text = f'Разница: {((war_mean - peace_mean) / peace_mean * 100):+.1f}%' if peace_mean
!= 0 else ''
        # Вывод статистики в консоль
print(f"\n{title}:")
if col_name == 'isk_velocity':
print(f"  Мирныепериоды: {len(peace_data)} записей, Среднее: {peace_mean:.4f}")
print(f"  Военныепериоды: {len(war_data)} записей, Среднее: {war_mean:.4f}")
        else:
print(f"  Мирныепериоды: {len(peace_data)} записей, Среднее: {peace_mean/1e12:.2f}
трлн")
print(f"  Военныепериоды: {len(war_data)} записей, Среднее: {war_mean/1e12:.2f} трлн")
ax.text(0.02, 0.98, stat_text, transform=ax.transAxes,
fontsize=10, verticalalignment='top',
bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
        if diff_text:
ax.text(0.02, 0.85, diff_text, transform=ax.transAxes,
fontsize=9, verticalalignment='top',
bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.5))
plt.tight_layout()
        if save_path:
output_path = Path(save_path) / f'boxplot_{col_name}.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
print(f"Боксплот {title} сохраненв: {output_path}")
plt.show()

self.results['boxplot_comparison'] = comparison_metrics
    return None
  def plot_correlation_matrix(self, save_path=None):
"""
    Построение тепловой карты корреляций между показателями
    """
print("\nАнализ корреляционных взаимосвязей между показателями...")
    # Выбор показателей для корреляционного анализа
correlation_cols = [
'total_isk_destroyed', 'production_isk', 'trade_value',
        'isk_velocity', 'mining_isk'
    ]

    # Фильтрациядоступныхстолбцов
available_cols = [col for col in correlation_cols if col in self.df.columns]

iflen(available_cols) < 2:
```

```
print("Недостаточно данных для корреляционного анализа")
return None
        # Расчеткорреляционнойматрицы
corr_matrix = self.df[available_cols].corr(method='spearman')
# Перевод названий на русский для графика
russian_names = {
            'total_isk_destroyed': 'Боевыепотери',
            'production_isk': 'Производство',
            'trade_value': 'Объемторговли',
            'isk_velocity': 'Скоростьобращения',
            'mining_isk': 'Добычаресурсов'
        }
display_names = [russian_names.get(col, col) for col in available_cols]
# Создание тепловой карты
fig, ax = plt.subplots(figsize=(12, 10))
        # Маска для верхнего треугольника
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
        # Построение тепловой карты
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.3f', cmap='RdBu_r',
center=0, square=True, linewidths=1, cbar_kws={"shrink": 0.8},
xticklabels=display_names, yticklabels=display_names, ax=ax,
annot_kws={"size": 11, "weight": "bold"})
ax.set_title('Матрица корреляций Спирмена между ключевыми показателями',
fontsize=16, fontweight='bold', pad=20)
plt.tight_layout()
        if save_path:
output_path = Path(save_path) / 'correlation_matrix.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
print(f"Корреляционная матрица сохранена в: {output_path}")
plt.show()
# Анализкорреляцийсбоевымипотерями
if 'total_isk_destroyed' in corr_matrix.columns:
war_correlations = corr_matrix['total_isk_destroyed'].sort_values(ascending=False)
print("\nКорреляции с показателем боевых потерь:")
for idx, (indicator, corr) in enumerate(war_correlations.items(), 1):
                if indicator != 'total_isk_destroyed':
indicator_name = russian_names.get(indicator, indicator)
                    strength = "сильная" if abs(corr) > 0.7 else "средняя" if abs(corr) > 0.3 else
"слабая"
sign = "положительная" ifcorr> 0 else "отрицательная"
print(f"{idx:2}. {indicator_name:25} r = {corr:.3f} ({sign} {strength} связь)")
# Анализ сильных корреляций
print("\nСильные корреляции (|r| > 0.7):")
strong_corrs = []
for i in range(len(corr_matrix.columns)):
            for j in range(i+1, len(corr_matrix.columns)):
corr = corr_matrix.iloc[i, j]
                if abs(corr) > 0.7:
                    col1 = russian_names.get(corr_matrix.columns[i], corr_matrix.columns[i])
                    col2 = russian_names.get(corr_matrix.columns[j], corr_matrix.columns[j])
strong_corrs.append((col1, col2, corr))
        if strong_corrs:
```

```
            for idx, (col1, col2, corr) in enumerate(strong_corrs, 1):
                print(f"{idx:2}. {col1:25} ↔ {col2:25} r = {corr:.3f}")
else:
print("  Нет сильных корреляций")
self.results['correlation_matrix'] = corr_matrix
        return corr_matrix
    def analyze_war_peace_statistics(self):
"""
        Детальный статистический анализ различий между военными и мирными периодами
"""
        # Разделениеданных
war_df = self.df[self.df['is_war_period'] == 1]
peace_df = self.df[self.df['is_war_period'] == 0]

print(f"Военные периоды: {len(war_df)} месяцев")
print(f"Мирные периоды: {len(peace_df)} месяцев")
print(f"Доля военных периодов: {len(war_df)/len(self.df)*100:.1f}%\n")
# Показателидлясравнения
comparison_metrics = ['production_isk', 'trade_value', 'isk_velocity', 'mining_isk']
comparison_results = []
        for metric in comparison_metrics:
            if metric not in self.df.columns:
                continue
            # Статистикипогруппам
war_mean = war_df[metric].mean()
peace_mean = peace_df[metric].mean()
war_std = war_df[metric].std()
peace_std = peace_df[metric].std()
            # Относительнаяразница
relative_diff = ((war_mean - peace_mean) / peace_mean * 100) if peace_mean != 0 else 0
            # Форматированиезначений
            if metric == 'isk_velocity':
war_mean_fmt = f"{war_mean:.4f}"
peace_mean_fmt = f"{peace_mean:.4f}"
war_std_fmt = f"{war_std:.4f}"
peace_std_fmt = f"{peace_std:.4f}"
            else:
war_mean_fmt = f"{war_mean/1e12:.2f} трлн"
peace_mean_fmt = f"{peace_mean/1e12:.2f} трлн"
war_std_fmt = f"{war_std/1e12:.2f} трлн"
peace_std_fmt = f"{peace_std/1e12:.2f} трлн"
# Название показателя на русском
russian_names = {
                'production_isk': 'Производство',
                'trade_value': 'Объем торговли',
                'isk_velocity': 'Скорость обращения',
                'mining_isk': 'Добыча ресурсов'
}
            result = {
                'Показатель': russian_names.get(metric, metric),
'Среднее (война)': war_mean_fmt,
                'Среднее (мир)': peace_mean_fmt,
```

```
'Разница, %': f"{relative_diff:+.1f}%",
            'σ (война)': war_std_fmt,
            'σ (мир)': peace_std_fmt
        }
comparison_results.append(result)
    # Выводрезультатов
comparison_df = pd.DataFrame(comparison_results)
    print(comparison_df.to_string(index=False))
# Анализстатистическойзначимости
    for metric in comparison_metrics:
        if metric not in self.df.columns:
            continue
war_mean = war_df[metric].mean()
peace_mean = peace_df[metric].mean()
relative_diff = ((war_mean - peace_mean) / peace_mean * 100) if peace_mean != 0 else 0
russian_name = {
            'production_isk': 'производства',
            'trade_value': 'объематорговли',
'isk_velocity': 'скорости обращения денег',
'mining_isk': 'добычиресурсов'
}.get(metric, metric)
        if abs(relative_diff) > 10:
            direction = "выше" if relative_diff> 0 else "ниже"
print(f"• Ввоенныепериоды {russian_name} всреднемна {abs(relative_diff):.1f}% {direction},
чемвмирные")
self.results['war_peace_comparison'] = comparison_df
    return comparison_df
  def generate_summary_report(self, output_dir=None):
report_lines = []
# 1. Общаяинформация
report_lines.append("1. ОБЩАЯИНФОРМАЦИЯОДАННЫХ")
report_lines.append(f"   Периоданализа: {self.df['history_date'].min().date()} - "
                    f"{self.df['history_date'].max().date()}")
report_lines.append(f"   Всегомесяцев: {len(self.df)}")
war_months = (self.df['is_war_period'] == 1).sum()
war_percentage = war_months/len(self.df)*100
report_lines.append(f"   Военныемесяцы: {war_months} ({war_percentage:.1f}%)")
# 2. Ключевые наблюдения
report_lines.append("\n2. КЛЮЧЕВЫЕ НАБЛЮДЕНИЯ")
# Анаиз динамики
    if 'total_isk_destroyed' in self.df.columns:
max_war_month = self.df.loc[self.df['total_isk_destroyed'].idxmax()]
report_lines.append(f"   Пикбоевыхпотерь: {max_war_month['total_isk_destroyed']/1e12:.2f}
трлн ISK "
                    f"({max_war_month['history_date'].strftime('%Y-%m')})")
# 3. Предварительные выводы
report_lines.append("\n3. ПРЕДВАРИТЕЛЬНЫЕ ВЫВОДЫ")
# Проверкавизуальныхразличий
if 'production_isk' inself.df.columns:
war_production = self.df[self.df['is_war_period'] == 1]['production_isk'].mean()
peace_production = self.df[self.df['is_war_period'] == 0]['production_isk'].mean()
        if war_production>peace_production:
```

```
            diff = ((war_production - peace_production) / peace_production * 100)
report_lines.append(f"   • Производство в военные периоды выше на {diff:.1f}% (требует
статистической проверки)")
if 'trade_value' in self.df.columns:
war_trade = self.df[self.df['is_war_period'] == 1]['trade_value'].mean()
peace_trade = self.df[self.df['is_war_period'] == 0]['trade_value'].mean()
        if war_trade>peace_trade:
            diff = ((war_trade - peace_trade) / peace_trade * 100)
report_lines.append(f"   • Объем торговли в военные периоды выше на {diff:.1f}%")
    # Сохранение отчета
ifoutput_dir:
output_dir = Path(output_dir)
output_dir.mkdir(exist_ok=True)
report_path = output_dir / 'eda_summary_report.txt'
        with open(report_path, 'w', encoding='utf-8') as f:
f.write('\n'.join(report_lines))
print(f"\nОтчет сохранен в: {report_path}")
    # Вывод отчета в консоль
print('\n'.join(report_lines))
    return report_lines
def main():
# Пути к данным
data_path = Path(r"данные\Подготовленные данные\eve_fixed_velocity.csv")
output_dir = Path(r"данные\Результаты анализа")
  # Создание директории для результатов
output_dir.mkdir(exist_ok=True)
# Инициализацияанализатора
  analyzer = EveExploratoryAnalysis(data_path)
  try:
    # 1. Загрузкаданных
df = analyzer.load_and_prepare_data()
    # 2. Базовыестатистики
analyzer.calculate_basic_statistics()
# 3. Временные ряды (отдельные окна для каждого графика)
analyzer.plot_time_series_with_war_periods(save_path=output_dir)
# 4. Боксплоты сравнения (отдельные окна для каждого графика)
analyzer.plot_comparison_boxplots(save_path=output_dir)
# 5. Корреляционныйанализ (одноокно)
analyzer.plot_correlation_matrix(save_path=output_dir)
# 6. Детальныйстатистическийанализ
analyzer.analyze_war_peace_statistics()
    # 7. Сводныйотчет
analyzer.generate_summary_report(output_dir=output_dir)
print(f"Все графики и отчеты сохранены в: {output_dir.absolute()}")
except Exception as e:
print(f"\nОшибкапривыполнениианализа: {e}")
    import traceback
traceback.print_exc()
if __name__ == "__main__":
main()
```

ПРИЛОЖЕНИЕ В

Листинг программы на языке Python для проверки установленных гипотез

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')

class EveHypothesisTesting:
"""
    Класс для статистической проверки гипотез исследования
"""

    def __init__(self, data_path):
        """
Инициализациятестировщикагипотез
"""
self.data_path = Path(data_path)
self.df = None
self.results = {}

defload_data(self):
        """
        Загрузка данных для статистического анализа
        """
print("Загрузка данных для проверки гипотез...")

        # Загрузка данных
self.df = pd.read_csv(self.data_path)
self.df['history_date'] = pd.to_datetime(self.df['history_date'])

        # Сортировкаподате
self.df = self.df.sort_values('history_date')

print(f"Данныезагружены. Период: {self.df['history_date'].min().date()} - "
            f"{self.df['history_date'].max().date()}")
print(f"Всего наблюдений: {len(self.df)}")
print(f"Военныепериоды: {(self.df['is_war_period'] == 1).sum()}")
print(f"Мирныепериоды: {(self.df['is_war_period'] == 0).sum()}")

        return self.df

    def test_normality(self, data, test_name='Shapiro-Wilk'):
"""
        Проверка нормальности распределения данных
```

```
        """
iftest_name == 'Shapiro-Wilk':
# Тест Шапиро-Уилка (для n < 5000)
if len(data) < 5000:
            stat, p_value = stats.shapiro(data.dropna())
test_result = {
                'test': 'Shapiro-Wilk',
                'statistic': stat,
                'p_value': p_value,
'normal': p_value> 0.05
}
else:
            # Для больших выборок используем тест Колмогорова-Смирнова
stat, p_value = stats.kstest(data.dropna(), 'norm')
test_result = {
                'test': 'Kolmogorov-Smirnov',
                'statistic': stat,
                'p_value': p_value,
                'normal': p_value> 0.05
            }
        else:
           stat, p_value = stats.kstest(data.dropna(), 'norm')
test_result = {
            'test': 'Kolmogorov-Smirnov',
            'statistic': stat,
            'p_value': p_value,
            'normal': p_value> 0.05
           }

        return test_result

    def hypothesis_1_production_war_peace(self, alpha=0.05):
"""
        Проверка гипотезы 1: Средний объём производства в военные периоды выше
        """
print("\n" + "="*70)
print("ПРОВЕРКА ГИПОТЕЗЫ 1: Производство в военные периоды выше")
print("="*70)

        # Разделениеданных
war_production = self.df[self.df['is_war_period'] == 1]['production_isk']
peace_production = self.df[self.df['is_war_period'] == 0]['production_isk']

        # Описательныестатистики
war_stats = {
           'n': len(war_production),
           'mean': war_production.mean(),
           'std': war_production.std(),
           'median': war_production.median()
        }

peace_stats = {
```

```
            'n': len(peace_production),
            'mean': peace_production.mean(),
            'std': peace_production.std(),
            'median': peace_production.median()
        }

print(f"Военныепериоды (n={war_stats['n']}):")
print(f"  Среднее: {war_stats['mean']/1e12:.2f} трлн ISK")
print(f"  Медиана: {war_stats['median']/1e12:.2f} трлн ISK")
print(f"  Ст.отклонение: {war_stats['std']/1e12:.2f} трлн ISK")

print(f"\nМирныепериоды (n={peace_stats['n']}):")
print(f"  Среднее: {peace_stats['mean']/1e12:.2f} трлн ISK")
print(f"  Медиана: {peace_stats['median']/1e12:.2f} трлн ISK")
print(f"  Ст.отклонение: {peace_stats['std']/1e12:.2f} трлн ISK")

        # Относительнаяразница
relative_diff = ((war_stats['mean'] - peace_stats['mean']) / peace_stats['mean'] * 100)
print(f"\nОтносительная разница: {relative_diff:+.2f}%")

        # Проверка нормальности распределений
print("\nПроверканормальностираспределений:")

war_normality = self.test_normality(war_production)
peace_normality = self.test_normality(peace_production)

print(f"  Военныепериоды: {war_normality['test']} W={war_normality['statistic']:.4f},
p={war_normality['p_value']:.4f}")
print(f"    Нормальноераспределение: {'Да' if war_normality['normal'] else 'Нет'}")

print(f"  Мирныепериоды: {peace_normality['test']} W={peace_normality['statistic']:.4f},
p={peace_normality['p_value']:.4f}")
print(f"    Нормальноераспределение: {'Да' if peace_normality['normal'] else 'Нет'}")

# Выбор теста в зависимости от нормальности
if war_normality['normal'] and peace_normality['normal']:
# Проверка равенства дисперсий (тест Левена)
levene_stat, levene_p = stats.levene(war_production.dropna(), peace_production.dropna())
print(f"\nТестЛевенанаравенстводисперсий: F={levene_stat:.4f}, p={levene_p:.4f}")

if levene_p > 0.05:
            # Дисперсии равны - t-тест Стьюдента
print("\nИспользуем t-тест Стьюдента (дисперсии равны)")
t_stat, p_value = stats.ttest_ind(war_production, peace_production,
equal_var=True, nan_policy='omit')
test_used = "t-тест Стьюдента (равные дисперсии)"
else:
            # Дисперсии не равны - тест Уэлча
print("\nИспользуем t-тест Уэлча (дисперсии не равны)")
t_stat, p_value = stats.ttest_ind(war_production, peace_production,
equal_var=False, nan_policy='omit')
test_used = "t-тест Уэлча (неравные дисперсии)"
```

```
else:
        # Непараметрический тест - U-тест Манна-Уитни
print("\nИспользуем U-тест Манна-Уитни (данные не нормальны)")
u_stat, p_value = stats.mannwhitneyu(war_production.dropna(),
peace_production.dropna(),
                                     alternative='greater')  #Проверяем, что war > peace
test_used = "U-тестМанна-Уитни"
t_stat = u_stat

    # Результаты теста
print(f"\nРезультаты статистического теста ({test_used}):")
print(f"  Статистика: {t_stat:.4f}")
print(f"  p-value: {p_value:.6f}")
print(f"  Уровень значимости (α): {alpha}")

    # Интерпретация результатов
if p_value <alpha:
print(f"\nРазличия статистически значимы (p < {alpha})")
print(f"  Производство в военные периоды ДЕЙСТВИТЕЛЬНО выше")
hypothesis_supported = True
    else:
print(f"Различиястатистическинезначимы (p ≥ {alpha})")
hypothesis_supported = False

    # Расчетразмераэффекта
    if test_used.startswith("t-тест"):
        # Коэна d для t-теста
pooled_std = np.sqrt(((war_stats['n']-1)*war_stats['std']**2 +
                      (peace_stats['n']-1)*peace_stats['std']**2) /
                     (war_stats['n'] + peace_stats['n'] - 2))
cohens_d = (war_stats['mean'] - peace_stats['mean']) / pooled_std
effect_size = cohens_d
effect_size_name = "Коэна d"
    else:
        # r для U-теста
        n1, n2 = war_stats['n'], peace_stats['n']
        r = 1 - (2 * u_stat) / (n1 * n2)
effect_size = r
effect_size_name = "r (U-тест)"

print(f"\nРазмерэффекта ({effect_size_name}): {effect_size:.4f}")

# Интерпретация размера эффекта
ifabs(effect_size) < 0.2:
effect_interpretation = "оченьманенький"
elif abs(effect_size) < 0.5:
effect_interpretation = "маленький"
elif abs(effect_size) < 0.8:
effect_interpretation = "средний"
    else:
effect_interpretation = "большой"
print(f"  Интерпретация: {effect_interpretation} эффект")
```

```
# Сохранение результатов
result = {
        'hypothesis': 'Гипотеза 1: Производство выше в военные периоды',
'test_used': test_used,
        'test_statistic': t_stat,
        'p_value': p_value,
        'alpha': alpha,
        'significant': hypothesis_supported,
        'war_stats': war_stats,
        'peace_stats': peace_stats,
        'relative_difference_percent': relative_diff,
        'effect_size': effect_size,
        'effect_size_name': effect_size_name,
        'effect_interpretation': effect_interpretation,
        'war_normality': war_normality,
        'peace_normality': peace_normality
    }
self.results['hypothesis_1'] = result
    return result
  def hypothesis_2_trade_correlation(self, alpha=0.01):
"""
    Проверка гипотезы 2: Корреляция торговли и скорости обращения с боевыми
потерями
    """
print("\n" + "="*70)
print("ПРОВЕРКА ГИПОТЕЗЫ 2: Корреляция с боевыми потерями")
print("="*70)
results = {}
    # Показатели для проверки
indicators = [
        ('trade_value', 'Объем торговли'),
        ('isk_velocity', 'Скорость обращения денег')
]
    for indicator, indicator_name in indicators:
print(f"\n--- {indicator_name} ---")
        # Удаляемпропущенныезначения
valid_data = self.df[['total_isk_destroyed', indicator]].dropna()
        if len(valid_data) < 2:
print(f"  Недостаточноданныхдляанализа {indicator_name}")
            continue

        # РасчеткорреляцииСпирмена
        correlation, p_value = stats.spearmanr(valid_data['total_isk_destroyed'],
valid_data[indicator])

print(f"  КоэффициенткорреляцииСпирмена (ρ): {correlation:.4f}")
print(f"  p-value: {p_value:.6f}")

        # Интерпретациясилыкорреляции
abs_corr = abs(correlation)
        if abs_corr< 0.3:
            strength = "слабая"
```

```
elif abs_corr < 0.7:
            strength = "средняя"
        else:
            strength = "сильная"
        # Знаккорреляции
        sign = "положительная" if correlation > 0 else "отрицательная"
print(f"  Силасвязи: {strength} {sign} связь")

# Проверка статистической значимости
if p_value <alpha:
print(f"Корреляция статистически значима (p < {alpha})")
significant = True
else:
print(f"Корреляция статистически не значима (p ≥ {alpha})")
significant = False

        # Дополнительно: линейная регрессия
print(f"\n  Линейная регрессия:")
try:
            X = valid_data['total_isk_destroyed']
y = valid_data[indicator]

            # Простая линейная регрессия
slope, intercept, r_value, p_reg, std_err = stats.linregress(X, y)

print(f"    Коэффициентдетерминации (R²): {r_value**2:.4f}")
print(f"    Наклон (slope): {slope:.6f}")
print(f"    p-value регрессии: {p_reg:.6f}")

if p_reg < alpha:
print(f"Регрессия статистически значима")
else:
print(f"Регрессия статистически не значима")
regression_result = {
                'r_squared': r_value**2,
                'slope': slope,
                'p_regression': p_reg,
                'significant_regression': p_reg < alpha
            }
        except Exception as e:
print(f"    Ошибкаврегрессии: {e}")
regression_result = None
        # Сохранениерезультатов
        results[indicator] = {
            'indicator_name': indicator_name,
            'correlation_coefficient': correlation,
            'p_value': p_value,
            'significant': significant,
            'strength': strength,
            'sign': sign,
            'regression': regression_result
        }
```

```
self.results['hypothesis_2'] = results
        return results
    def hypothesis_3_mining_correlation(self, alpha=0.01):
"""
        Проверка гипотезы 3: Корреляция добычи ресурсов с активностью игроков
        """
print("\n" + "="*70)
print("ПРОВЕРКА ГИПОТЕЗЫ 3: Корреляция добычи с активностью игроков")
print("="*70)
        # Используем total_isk_destroyed как показатель активности игроков
indicator = 'mining_isk'
indicator_name = 'Добычаресурсов'
activity_indicator = 'total_isk_destroyed'
print(f"Показательактивности: {activity_indicator} (боевыепотери)")
print(f"Целевой показатель: {indicator_name}")
        # Удаляем пропущенные значения
valid_data = self.df[[activity_indicator, indicator]].dropna()
iflen(valid_data) < 2:
print("Недостаточно данных для анализа")
return None
print(f"Количествонаблюдений: {len(valid_data)}")
        # РасчеткорреляцииСпирмена
        correlation, p_value = stats.spearmanr(valid_data[activity_indicator],
valid_data[indicator])
print(f"\nКоэффициенткорреляцииСпирмена (ρ): {correlation:.4f}")
print(f"p-value: {p_value:.6f}")
        # Интерпретациясилыкорреляции
abs_corr = abs(correlation)
        if abs_corr< 0.3:
            strength = "слабая"
elif abs_corr < 0.7:
            strength = "средняя"
        else:
            strength = "сильная"
        # Знаккорреляции
        sign = "положительная" if correlation > 0 else "отрицательная"
print(f"Силасвязи: {strength} {sign} связь")
# Проверка статистической значимости
if p_value <alpha:
print(f"Корреляция статистически значима (p < {alpha})")
significant = True
else:
print(f"Корреляция статистически не значима (p ≥ {alpha})")
significant = False
        # Дополнительно: линейная регрессия с контролем времени
print(f"\nРасширенный анализ (регрессия с контролем тренда):")
try:
            # Создаем временной тренд
temp_df = valid_data.copy()
temp_df['time_trend'] = range(len(temp_df))
            # Множественнаярегрессия
            model = ols(f'{indicator} ~ {activity_indicator} + time_trend', data=temp_df).fit()
```

```
print(f"  Множественнаярегрессия:")
print(f"    R²: {model.rsquared:.4f}")
print(f"    Скорректированный R²: {model.rsquared_adj:.4f}")
            # Коэффициенты
            params = model.params
pvalues = model.pvalues
print(f"\n  Коэффициенты регрессии:")
for param in params.index:
pval = pvalues[param]
                sig = "" if pval< alpha else ""
print(f"    {param}: {params[param]:.6f} (p={pval:.6f}) {sig}")

            # Интерпретациявлияниясактивности
activity_coef = params[activity_indicator]
activity_p = pvalues[activity_indicator]

if activity_p <alpha:
print(f"\nВлияние активности игроков статистически значимо")
print(f"    При увеличении боевых потерь на 1 трлн ISK,")
print(f"    добыча ресурсов изменяется на {activity_coef/1e12:.4f} трлн ISK")
else:
print(f"\n Влияние активности игроков статистически не значимо")
regression_result = {
                'r_squared': model.rsquared,
                'adj_r_squared': model.rsquared_adj,
                'coefficients': params.to_dict(),
                'p_values': pvalues.to_dict(),
                'f_statistic': model.fvalue,
                'f_pvalue': model.f_pvalue,
                'significant_f': model.f_pvalue< alpha
            }
        except Exception as e:
print(f"  Ошибкаврегрессионномманализе: {e}")
regression_result = None
        # Сохранениерезультатов
        result = {
            'indicator_name': indicator_name,
            'activity_indicator': activity_indicator,
            'correlation_coefficient': correlation,
            'p_value': p_value,
            'significant': significant,
            'strength': strength,
            'sign': sign,
            'regression': regression_result
        }
self.results['hypothesis_3'] = result
        return result
    def generate_summary_report(self, output_dir=None):
"""
        Генерация сводного отчета по результатам проверки гипотез
"""
report_lines = []
```

```
        # 1. Гипотеза 1
        if 'hypothesis_1' in self.results:
            h1 = self.results['hypothesis_1']
report_lines.append("\n1. ГИПОТЕЗА 1: Производство в военные периоды выше")
report_lines.append("-"*50)
report_lines.append(f"   Использованныйтест: {h1['test_used']}")
report_lines.append(f"   Статистикатеста: {h1['test_statistic']:.4f}")
report_lines.append(f"   p-value: {h1['p_value']:.6f}")
report_lines.append(f"   Уровеньзначимости (α): {h1['alpha']}")
if h1['significant']:
report_lines.append("статистическизначимыеразличия")

report_lines.append(f"   Относительнаяразница: {h1['relative_difference_percent']:+.2f}%")
report_lines.append(f"   Размерэффекта ({h1['effect_size_name']}): {h1['effect_size']:.4f}")
report_lines.append(f"   Интерпретацияэффекта: {h1['effect_interpretation']}")

        # 2. Гипотеза 2
        if 'hypothesis_2' in self.results:
report_lines.append("\n2. ГИПОТЕЗА 2: Корреляция с боевыми потерями")
report_lines.append("-"*50)
            h2 = self.results['hypothesis_2']
            for indicator, result in h2.items():
indicator_name = result['indicator_name']
report_lines.append(f"   Показатель: {indicator_name}")
report_lines.append(f"     КоэффициенткорреляцииСпирмена:
{result['correlation_coefficient']:.4f}")
report_lines.append(f"     p-value: {result['p_value']:.6f}")
report_lines.append(f"     Силасвязи: {result['strength']} {result['sign']}")
                if result['significant']:
report_lines.append("Корреляциястатистическизначима")
else:
report_lines.append("Корреляция статистически не значима")
if result['regression']:
report_lines.append(f"     R² линейнойрегрессии: {result['regression']['r_squared']:.4f}")
        # 3. Гипотеза 3
        if 'hypothesis_3' in self.results:
            h3 = self.results['hypothesis_3']
report_lines.append("\n3. ГИПОТЕЗА 3: Добыча ресурсов и активность игроков")
report_lines.append("-"*50)
report_lines.append(f"   Показательактивности: {h3['activity_indicator']}")
report_lines.append(f"   КоэффициенткорреляцииСпирмена:
{h3['correlation_coefficient']:.4f}")
report_lines.append(f"   p-value: {h3['p_value']:.6f}")
report_lines.append(f"   Силасвязи: {h3['strength']} {h3['sign']}")
            if h3['significant']:
report_lines.append("Корреляциястатистическизначима")
if h3['regression']:
report_lines.append(f"   R² множественнойрегрессии: {h3['regression']['r_squared']:.4f}")
report_lines.append(f"   Скорректированный R²: {h3['regression']['adj_r_squared']:.4f}")
# 4. Общиевыводы
        # Подсчет подтвержденных гипотез
confirmed = 0
```

```
total = 0
        if 'hypothesis_1' in self.results:
            total += 1
            if self.results['hypothesis_1']['significant']:
                confirmed += 1
        if 'hypothesis_2' in self.results:
            h2 = self.results['hypothesis_2']
            for result in h2.values():
                total += 1
                if result['significant']:
                    confirmed += 1
        if 'hypothesis_3' in self.results:
            total += 1
            if self.results['hypothesis_3']['significant']:
                confirmed += 1
report_lines.append(f"Подтвержденогипотез: {confirmed} из {total}")
# Сохранение отчета
ifoutput_dir:
output_dir = Path(output_dir)
output_dir.mkdir(exist_ok=True)
report_path = output_dir / 'hypothesis_testing_report.txt'
            with open(report_path, 'w', encoding='utf-8') as f:
f.write('\n'.join(report_lines))
print(f"\nОтчет сохранен в: {report_path}")
        # Вывод отчета в консоль
print('\n'.join(report_lines))
        return report_lines
    def save_detailed_results(self, output_dir):
"""
        Сохранение детальных результатов в формате CSV
"""
output_dir = Path(output_dir)
output_dir.mkdir(exist_ok=True)

        # 1. Результатыгипотезы 1
        if 'hypothesis_1' in self.results:
            h1 = self.results['hypothesis_1']
# Создаем DataFrame с результатами
            h1_data = {
                'Параметр': [
                    'Использованный тест',
                    'Статистика теста',
                    'p-value',
                    'Уровень значимости (α)',
                    'Статистическая значимость',
                    'Относительная разница (%)',
                    'Размер эффекта',
                    'Тип размера эффекта',
                    'Интерпретация эффекта'
                ],
                'Значение': [
                    h1['test_used'],
```

```
f"{h1['test_statistic']:.4f}",
                f"{h1['p_value']:.6f}",
                str(h1['alpha']),
                'Да' if h1['significant'] else 'Нет',
                f"{h1['relative_difference_percent']:+.2f}",
                f"{h1['effect_size']:.4f}",
                h1['effect_size_name'],
                h1['effect_interpretation']
            ]
        }
        h1_df = pd.DataFrame(h1_data)
        h1_path = output_dir / 'hypothesis_1_results.csv'
        h1_df.to_csv(h1_path, index=False, encoding='utf-8-sig')
print(f"Результаты гипотезы 1 сохранены в: {h1_path}")
# 2. Результатыгипотезы 2
    if 'hypothesis_2' in self.results:
        h2 = self.results['hypothesis_2']
        h2_data = []
        for indicator, result in h2.items():
            h2_data.append({
                'Показатель': result['indicator_name'],
                'КорреляцияСпирмена': f"{result['correlation_coefficient']:.4f}",
                'p-value': f"{result['p_value']:.6f}",
                'Статистическаязначимость': 'Да' if result['significant'] else 'Нет',
                'Силасвязи': result['strength'],
                'Направление': result['sign'],
                'R² регрессии': f"{result['regression']['r_squared']:.4f}" if result['regression'] else
'N/A'
            })
        h2_df = pd.DataFrame(h2_data)
        h2_path = output_dir / 'hypothesis_2_results.csv'
        h2_df.to_csv(h2_path, index=False, encoding='utf-8-sig')
print(f"Результаты гипотезы 2 сохранены в: {h2_path}")
# 3. Результатыгипотезы 3
    if 'hypothesis_3' in self.results:
        h3 = self.results['hypothesis_3']
h3_data = {
            'Параметр': [
                'Показатель активности',
                'Целевой показатель',
                'Корреляция Спирмена',
                'p-value',
                'Статистическая значимость',
                'Сила связи',
                'Направление',
                'R² регрессии',
                'Скорректированный R²'
            ],
'Значение': [
                h3['activity_indicator'],
                h3['indicator_name'],
                f"{h3['correlation_coefficient']:.4f}",
```

```
                f"{h3['p_value']:.6f}",
                'Да' if h3['significant'] else 'Нет',
                h3['strength'],
                h3['sign'],
                f"{h3['regression']['r_squared']:.4f}" if h3['regression'] else 'N/A',
                f"{h3['regression']['adj_r_squared']:.4f}" if h3['regression'] else 'N/A'
            ]
        }
        h3_df = pd.DataFrame(h3_data)
        h3_path = output_dir / 'hypothesis_3_results.csv'
        h3_df.to_csv(h3_path, index=False, encoding='utf-8-sig')
print(f"Результаты гипотезы 3 сохранены в: {h3_path}")
defmain():
    """
    Основная функция для запуска проверки гипотез
    """
    # Пути к данным
data_path = Path(r"данные\Подготовленные данные\eve_fixed_velocity.csv")
output_dir = Path("“данные\Результаты анализа")
    # Создание директории для результатов
output_dir.mkdir(exist_ok=True)
# Инициализациятестировщика
    tester = EveHypothesisTesting(data_path)
    try:
        # 1. Загрузкаданных
df = tester.load_data()
        # 2. Проверкагипотезы 1
        result_h1 = tester.hypothesis_1_production_war_peace(alpha=0.05)

        # 3. Проверкагипотезы 2
        result_h2 = tester.hypothesis_2_trade_correlation(alpha=0.01)

        # 4. Проверкагипотезы 3
        result_h3 = tester.hypothesis_3_mining_correlation(alpha=0.01)
        # 5. Генерациясводногоотчета
tester.generate_summary_report(output_dir=output_dir)
        # 6. Сохранениедетальныхрезультатов
tester.save_detailed_results(output_dir)
    except Exception as e:
print(f"\nОшибка при проверке гипотез: {e}")
import traceback
traceback.print_exc()
if __name__ == "__main__":
main()
```